

Explicit Credit Assignment through Local Rewards and Dependence Graphs

Bang Giang Le¹ Viet Cuong Ta^{1*}

¹Human-Machine Interaction Laboratory
VNU University of Engineering and Technology, Hanoi, Vietnam
{giangbang, cuongtv}@vnu.edu.vn

Abstract

To promote cooperation in Multi-Agent Reinforcement Learning, the reward signals of all agents can be aggregated together, forming global rewards that are commonly known as the fully cooperative setting. However, global rewards are usually noisy because they contain the contributions of all agents, which have to be resolved in the credit assignment process. On the other hand, using local reward benefits from faster learning due to the separation of agents' contributions, but can be suboptimal as agents myopically optimize their own reward while disregarding the global optimality. In this work, we propose a method that combines the merits of both approaches. By using a dependence graph between agents' interaction, our method discerns the individual agent contribution in a more fine-grained manner than a global reward, while alleviating the cooperation problem with agents' local reward. We also introduce a practical approach for approximating such a graph. We test the proposed approach with known dependence graphs and learned dependence graphs in fully cooperative settings. The results demonstrate the flexibility of the approach, enabling improvements over the traditional local and global reward settings.

1 Introduction

Cooperation between multiple agents in Multi-Agent Reinforcement Learning (MARL) can be understood as the collective effort whereby multiple agents jointly pursue a shared objective, thereby achieving a better outcome that might be unattainable by individual agents alone. Such collaboration is crucial in real-world applications, where tasks such as autonomous driving Shalev-Shwartz et al. (2016), distributed voltage control Wang et al. (2021), or swarm coordination Cao et al. (2012) are often too complex for isolated agents to solve effectively. As a result, Cooperative MARL remains a central research direction within multi-agent learning Yuan et al. (2023).

The simplest approach to enforce cooperation within a group of agents is through reward aggregation (scalarization), where the rewards of individuals are replaced by the sum (average) of the agents' rewards in the whole group Samvelyan et al. (2019); Lowe et al. (2017). In this way, all agents share a common objective, which is the reward of the whole group; the loss of the group is the loss of every agent. This setting generally goes under the name *fully cooperative* settings in the literature Yu et al. (2022); Rashid et al. (2020). Rewards scalarization, however, introduces the problem of *credit assignment* Chang et al. (2003); Foerster et al. (2018), where the information on the contributions of each agent is lost through the scalarization operations.

Alternatively, in a general *cooperative* setting, individual rewards are kept separate, and each agent optimizes its own objective independently, which effectively reduces the Multi-agent problem to a set of single-agent RL tasks De Witt et al. (2020). This learning approach is often referred to

*Corresponding author.

as *independent learning*, with examples including IQL Tan (1993), IPPO De Witt et al. (2020), and MADDPG Lowe et al. (2017). In this approach, each agent optimizes its own reward signal myopically, and therefore, agents only cooperate when cooperation is beneficial to them. In this case, cooperation can only emerge implicitly Zheng et al. (2018). Learning from local rewards can potentially lead to selfish behaviors that can be suboptimal to the entire system Devlin et al. (2014); Le and Ta (2025).

In fully cooperative MARL, it is often assumed that all agents share a single global reward. To mitigate the resulting credit assignment problem, various methods have been proposed, such as value decomposition Sunehag et al. (2017) and counterfactual learning Foerster et al. (2018). However, in many scenarios, the team’s reward can be naturally attributed to the contributions of specific subsets of agents, in which case local rewards can provide more fine-grained information about overall performance. Consequently, aggregating these signals into a single scalar and then attempting to recover them appears inherently inefficient. Motivated by this observation, we ask

Can we directly leverage the individual-agent reward information to effectively avoid the credit assignment issues introduced by reward scalarization, while still promoting cooperation?

Building on the framework of dependence graphs and Multi-Agent Networked MDPs (Qu et al., 2020), we combine local reward signals in a way that respects the credit of each agent. Our main contributions in this work are three-fold. First, we propose a novel policy gradient estimator that truncates irrelevant local rewards based on interaction paths in the dependence graph, effectively alleviating the credit assignment problem. Second, we prove that our method provides a smooth transition between the local and global reward learning depending on the density of the dependence graphs in terms of sample complexity. Third, we introduce a practical method for approximating the dependence graph based on our theory. Empirically, we validate the proposed approach on a range of benchmarks in MARL. Experimental results show that our methods can be competitive with and outperform strong baselines from both local and global reward settings.

2 Problem Setting and Notation

Markov Game. We formulate the MARL problem as a Markov game, defined by the tuple $\mathcal{M} = \langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathbf{P}, \mathbf{r}, \gamma \rangle$. Here, $\mathcal{N} = \{1, 2, \dots, N\}$ denotes the set of N agents, $\mathcal{S}$ is the state space, and $\mathcal{A} = \mathcal{A}_1 \times \dots \times \mathcal{A}_N$ represents the joint action space, where $\mathcal{A}_i$ is the action space of agent i . The function $\mathbf{P}$ defines the transition dynamics, $\mathbf{r} : \mathcal{S} \times \mathcal{A} \rightarrow [0, R_{\max}]^N$ is the joint reward function, and $\gamma \in [0, 1)$ is the discount factor. Throughout the paper, we use bold notation to indicate joint quantities: for example, the joint action at time step t is denoted by $\mathbf{a}_t = (a_t^1, \dots, a_t^N)$, and the joint policy by $\boldsymbol{\pi} = (\pi_1, \dots, \pi_N)$. At each time step t , all agents *independently* select actions according to their policies, forming the joint action $\mathbf{a}_t$, upon which the environment transitions to the next state s_{t+1} according to the transition kernel $\mathbf{P}$. Additionally, let $\rho_{\boldsymbol{\pi}}(\mathbf{s}) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\boldsymbol{\pi}}(\mathbf{s}_t = \mathbf{s})$ be the discounted marginal state distribution of policy $\boldsymbol{\pi}$.

Networked Multi-Agent MDP. Many practical MARL problems exhibit local dependence structures, where agents may interact with a small subset of neighbors and are relatively independent of other outsider agents. To facilitate such structural dependency, we utilize the framework of networked Markov systems (Bagnell and Ng, 2005; Qu and Li, 2019; Qu et al., 2020). More specifically, we assume that the joint state space $\mathcal{S}$ can be decomposed into substate spaces of individual agents, i.e., $\mathcal{S} = \mathcal{S}_1 \times \dots \times \mathcal{S}_N$, and an agent can only observe their respective local states, $\pi_i : \mathcal{S}_i \rightarrow \Delta(\mathcal{A}_i)$. The transition kernel $\mathbf{P}$ also factorizes accordingly: the next state of agent i depends only on a local agent-dependence set, denoted by $\text{Pa}^i(s^i) \subseteq [N]$, with

$$P_i(s^{i'} | \mathbf{s}, \mathbf{a}) = P_i(s^{i'} | \{s^k, a^k : k \in \text{Pa}^i(s^i)\}) \in \Delta(\mathcal{S}_i).$$

Note that the agent dependence set depends on the local states; agents can infer from their observations which other agents can affect their next state. We assume $i \in \text{Pa}^i(s^i)$, that is, each agent always influences its own next state. This decomposition structure naturally forms a (*state-*)*dependence graph*, which we define later in this section.

Each agent i also receives an individual reward $r^i : \mathcal{S}_i \times \mathcal{A}_i \rightarrow [0, R_{\max}]$ based on their states and actions. For clarity of exposition in the main text, we restrict to the case where r^i depends only on the

local state-action pair (s^i, a^i) . More general settings of the reward functions can be naturally captured by introducing a second *reward-dependence graph*. The reward-dependence graph captures myopic interactions: the immediate effect to current rewards, future propagation is already captured by the state-dependence graph. Since this extension does not alter the main theory and can be incorporated with minor modifications, we defer its full formalization to Appendix C.1.

We define the cumulative returns of agent i under the joint policy π as $J^i(\pi) = \mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_t^i]$. In the fully cooperative setting, agents optimize a shared objective by maximizing the team’s total expected return, $J(\pi) = \sum_{i=1}^N J^i(\pi)$, with $r_t = \sum_{i=1}^N r_t^i$ the scalarized reward used as a common learning signal for all agents. Throughout the paper, we refer to the scalarized reward r_t as *global reward* to distinguish it from the *local reward* r_t^i .

The factorization of the global states resembles the partial observability property, where each agent only observes parts of the underlying states. However, unlike partial observation, the transition kernel directly operates on the partial states. Furthermore, one can recover the standard formulation by duplicating the true states, $\mathbf{s} = (s, \dots, s)$, and by defining the transition kernel P_i to include all the agents in the set Pa_i . Our formulation differs from prior work in the underlying dependence structure. In Qu and Li (2019), the dependencies form a tree, which corresponds to a dependence set of size one in our framework. Other works (Qu et al., 2020; Jing et al., 2024) consider a graph-structured formulation, but the graph is fixed, whereas in our setting, the dependence graph is dynamic and can vary across states, which admits static graphs as special cases.

State Dependence Graph. Based on the Dependence structure, we define the dependence graph, in which a directed edge exists between two agents if one agent influences the other’s next states.

Definition 2.1 (State Dependence Graph induced by a Networked Multi-Agent MDP). Let $\mathcal{M}$ be a Networked Multi-Agent MDP. The *State Dependence Graph* of $\mathcal{M}$ is a directed graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$, where $\mathcal{V} = \{(s, i) \mid s \in \mathcal{S}, i \in [N]\}$ is the vertex set and $\mathcal{E}$ is the edge set. A directed edge $(s, i) \rightarrow (s', j)$ belongs to $\mathcal{E}$ if $i \in \text{Pa}_j(s^j)$ and $\exists \mathbf{a} \in \mathcal{A}$ such that $\mathbf{P}(s' | \mathbf{s}, \mathbf{a}) > 0$.

Definition 2.2 (Proper Dependence Graph). A graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$ is proper if there exists a transition kernel $\mathbf{P}$ that admits it as a dependence graph; i.e. if $(s_1, i) \rightarrow (s'_1, j)$ and $(s_2, i) \rightarrow (s'_2, i)$ are two edges of $\mathcal{G}$ such that $s_1^j = s_2^j$, then $(s_2, i) \rightarrow (s'_2, j)$ is also an edge of $\mathcal{G}$. Every proper dependence graph has a unique collection of agent-dependence sets $\{\text{Pa}_G^i(s^i)\}_i^N$ for each state $\mathbf{s}$.

Fix a realized state trajectory $\mathbf{s}_{0:\infty} = (\mathbf{s}_0, \mathbf{s}_1, \dots)$, the k -hop parent sets of agent i at timestep t are defined recursively as:

$$\text{Pa}^i(s_t^i, 0, \mathbf{s}_{0:t}) = \{i\}, \quad \text{Pa}^i(s_t^i, k, \mathbf{s}_{0:t}) = \bigcup_{m \in \text{Pa}_i(s_t^i, k-1, \mathbf{s}_{0:t})} \text{Pa}^m(s_{t-k}^m), \quad k \geq 1.$$

The *first meeting time* between agent j starting from timestep t to the agent i in a trajectory $\mathbf{s}_{0:\infty}$ in a future timestep $t' > t$ is defined as

$$T_{ji}(s_t^j, \mathbf{s}_{0:\infty}, \mathcal{G}) := \min\{t' \geq t : j \in \text{Pa}_i(s_{t'}^i, t' - t, \mathbf{s}_{0:t'})\} - t, \quad (1)$$

with $T_{ij}(\cdot) = \infty$ if no such t' exists. We also write $T_{ji}(s)$ if the dependence graph and the trajectory are obvious from the context. The meeting time can be interpreted as the rate at which interactions between agents propagate across the graph.

Information-Theoretic Notation. We use capital letters to denote random variables corresponding to the same entities; for example, S^i and A^i represent the random variables associated with the (sub)state s^i and action a^i of the agent i . We also use $H(\cdot)$ to denote entropy and $I(\cdot; \cdot)$ to denote mutual information.

3 Motivation: Scalarize or not scalarize

Existing works in cooperative MARL either use a global reward, for example, in CTDE, or a local reward, such as in independent learning (see Section A). However, these two setups represent two extreme learning paradigms that are only suitable on a case-by-case basis. To illustrate, we provide an example of these two learning modes in the LBF environments (Christianos et al., 2020). We demonstrate two scenarios that exhibit contrasting learning patterns on the same backbone MAPPO algorithm. The results are presented in Figure (1). We observe the following:

On the one hand, learning with a global reward signal can ensure the cooperation of agents. Among the two scenarios, the global reward settings both converge. However, the variance of the estimated gradient in environments with a high number of agents can be significantly high; this additional variance is the cost of the credit assignment from scalarization. In fact, any global-reward algorithms face a theoretical sample-efficiency limit can scale with the number of agents Bagnell and Ng (2005), exacerbating the credit assignment problems (Foerster et al., 2018; Kuba et al., 2021). Existing works alleviate this problem either through adaptive trust region (Sun et al., 2022), sequential updates (Zhong et al., 2024), or conservative update mechanisms (Wu et al., 2021).

On the other hand, learning with separate reward signals can converge faster, especially in environments with a large number of agents, since local reward learning does not suffer from the credit assignment problem. However, the cost of this learning approach is the potential of suboptimal convergence Devlin et al. (2014); Le and Ta (2025). Resolving this problem generally requires careful reward engineering and experimentations (Mao et al., 2020).

We summarize our observation as the following *reward dilemma* for multi-agent RL

The credit assignment problem introduced by global reward scalarization imposes additional variance that scales with the number of agents, potentially slowing down the learning process. However, local reward learning can cause severe consequences due to miscoordination.

From this observation, we see the limitations of both the global scalarization and local reward learning. Restricting learning to only one of these settings can be insufficient for a truly general learning framework. This motivates the development of a framework that supports more flexible learning paradigms, allowing a smooth transition between these two extremes. In this work, we take a step in this direction. Our approach can offer greater flexibility to reward settings and facilitate more effective optimization in MARL, while mitigating the credit assignment issues of global rewards and the miscoordination in local rewards.

4 Method

4.1 Dependence Graph Policy Gradient

We now derive how the dependence structure introduced in the previous section enables a more efficient computation of the policy gradient. Our starting point is the fully cooperative objective, $J(\pi) = \sum_{i=1}^N J^i(\pi)$. The policy gradient with respect to agent j 's parameters can then be written as $\nabla_{\pi_j} J(\pi) = \sum_{i=1}^N \nabla_{\pi_j} J^i(\pi)$. Using the multi-agent policy gradient theorem (Foerster et al., 2018; Kuba et al., 2021), we have

$$\nabla_{\pi_j} J^i(\pi) = \frac{1}{(1-\gamma)} \mathbb{E}_{\rho_{\pi}, \pi} \left[\nabla \log \pi_j(a^j | s^j) Q_{\pi}^i(s, \mathbf{a}) \right], \quad (2)$$

where $Q_{\pi}^i(s, \mathbf{a})$ denotes the joint action-value function of agent i under the joint policy π . We include the derivation of (2) in appendix B.1 for completeness. In local reward settings, the cross-agent gradients are completely ignored; i.e. $\nabla_{\pi_i} J^j = 0, \forall j \neq i$.

We can further simplify the policy gradient in (2) to include only the necessary states that affect agent i 's rewards. As illustrated in Figure 2, if agent k takes an action at timestep t , it cannot directly affect the state of agent p at timestep $t+1$ as there is no edge connecting them. Therefore, the cross-gradient of agent k to p is zero in this case. However, the state of agent q at time $t+2$ can be influenced by

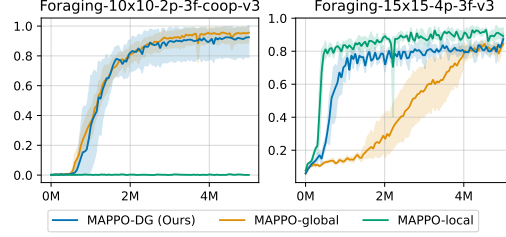


Figure 1: Reward dilemma in cooperative MARL; global reward enhances cooperation but introduces credit assignment problem, while local reward can induce suboptimal policies in the environments that require cooperation. Our method can enable faster training, as in local rewards, while avoiding the miscoordination pitfall.

agent k , but only through an intermediate state, which is an indirect effect that "propagates" from agent k through paths on the graph. We formalize this propagation effect in the following result.

Proposition 4.1. Fix a joint policy π . Let $i, j \in [N]$. The policy gradient $\nabla_{\pi_j} J^i(\pi)$ is given by

$$\frac{1}{1-\gamma} \mathbb{E}_{\mathbf{s}_0, \mathbf{a}_0 \sim \rho_{\pi}, \pi} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{\tau} \left[\sum_{k \geq T_{ji}(s_0^j, \tau)} \gamma^k r^i(s_k^i, a_k^i) \middle| \mathbf{s}_0, \mathbf{a}_0 \right] \right], \quad (3)$$

where $T_{ji}(s_0^j, \tau)$ is the first timestep first meeting time as defined in (1).

We call the policy gradient that is estimated through Proposition 4.1 the *dependency-graph* policy gradient, and denote it by $\nabla_{\pi_j} J(\pi; \mathcal{G})$. Compared to the traditional policy gradient, equation (3) differs by the cumulative reward that only starts after the timestep T_{ji} . As a result, if each agent operates relatively independently, i.e., a sparse dependence graph, then the cross-agent gradient can become exponentially small based on the meeting times. On the contrary, if all agents' MDPs are tightly coupled, e.g., a fully connected dependence graph, then (3) returns to the standard policy gradient.

Our dependency-graph Policy Gradient operates on an orthogonal dimension and is complementary to the Policy gradient with Networked MDPs in Qu and Li (2019); Qu et al. (2020). More specifically, compared to Lemma 2 in Qu et al. (2020), where the joint value functions are approximated through a small subset of k -hop parents, with the approximation error reducing exponentially fast with k , i.e., the value function *representation* dimension. Our policy gradient operates on the *time* dimension where the value functions can be estimated after the meeting timestep, without changing the gradient. Moreover, compared with the policy gradient theorems in Syed and Bai (2026) (Theorem 3.7) and Jing et al. (2024) (Lemma 2), our result generalizes the pruning scheme. Specifically, their theorems only prune agents that are unreachable from a given agent, corresponding to the special case $T_{ji} = \infty$ in our framework. As a result, their pruning mechanism is effective only when the graphs have multiple connected components.

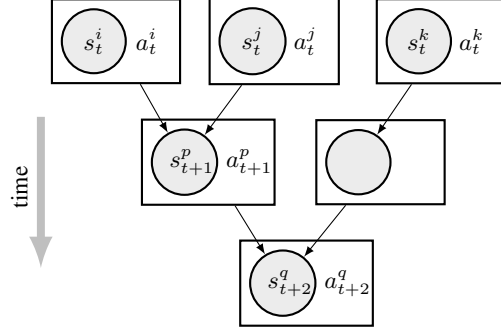


Figure 2: An example of an MDP with decomposed state structure. Agent k (top right) cannot influence agent q at timestep $t+1$; any effect can only occur from timestep $t+2$ onward (bottom). Since agent q lies in agent k 's blind spot, it can be excluded from the gradient computation at $t+1$. In this example, the meeting time $T_{kq}(s_t^k) = 2$.

4.2 Sample Complexity Analysis

In this section, we demonstrate sample complexity improvements of using the dependence graph through Policy Gradient (PG) algorithms. We let $\theta = (\theta_1, \theta_2, \dots, \theta_N)$ to denote the joint policy parameters, where each θ_i parameterises agent i 's policy. In the policy gradient framework, we estimate gradients from m sampled trajectories over a truncated horizon H . The standard estimator and its dependence-graph counterpart are given by:

$$\hat{\nabla}_{\theta_j}^m J^i(\theta) = \frac{1}{m} \sum_{l=1}^m \sum_{t=0}^H \gamma^t \nabla_{\theta_j} \log \pi_j(a_{t,l}^i | s_{t,l}^i) \sum_{k=t}^H \gamma^k r_{k,l}^i \quad (4)$$

$$\hat{\nabla}_{\theta_j}^m J^i(\theta, \mathcal{G}) = \frac{1}{m} \sum_{l=1}^m \sum_{t=0}^H \gamma^t \nabla_{\theta_j} \log \pi_j(a_{t,l}^j | s_{t,l}^j) \sum_{k=t+T_{ji}^H(s_{t,l}^j, \mathbf{s}_0, H, l)}^H \gamma^k r_{k,l}^i \quad (5)$$

With truncated dependence-graph gradient estimates, cross-agent gradients are dropped beyond horizon H ². Following gradient estimation, each agent j updates its parameters as $\theta_j^n = \theta_j^{n-1} + \eta \sum_i \hat{\nabla}_{\theta_j} J^i(\cdot)$, $\forall j \in [N]$, where η is the step size and θ_j^n is the agents' parameters at the iteration

²We define T_{ji}^H in (15).

n . This process repeats for a total of T iterations. Also, we note that the update in (4) corresponds to the GPOMDP estimator in the single-agent literature (Baxter and Bartlett, 2001).

Compared to the standard PG, the dependence-graph estimator (5) prunes timesteps prior to T_{ji} , yielding a reduction in effective sample weight by a factor of approximately $\gamma^{T_{ji}}$. To formalise this gain, we define

$$\Gamma_j^i := \sup_{k, \pi} \mathbb{E}[\gamma^{T_{ji}(s_k^j)}] \leq 1 \quad \Gamma_j := \sum_i \Gamma_j^i \leq N \quad \Gamma := \sum_j \Gamma_j \leq N^2,$$

The following result shows that exploiting the dependence structure leads directly to improved sample complexity.

Theorem 4.2. *Suppose that each agent’s policy satisfies the standard smoothness and Lipschitz condition (assumption D.1). Then, for a given $\epsilon > 0$, by choosing appropriate parameters of H, T, m, η (details in appendix D), we need a total number of samples of*

$$\begin{cases} O\left(\frac{NR_{\max}^4 \Gamma \max_j \Gamma_j \log(N/\epsilon)}{(1-\gamma)^6 \epsilon^4 \log(1/\gamma)}\right) & \text{using dependence graph PG (5)} \\ O\left(\frac{N^3 R_{\max}^4 \max_j \Gamma_j \log(N/\epsilon)}{(1-\gamma)^6 \epsilon^4 \log(1/\gamma)}\right) & \text{using standard PG (4)} \end{cases}$$

to guarantee convergence to an ϵ stationary point $\mathbb{E}[\|\nabla_{\theta^U} J(\theta^U)\|^2] \leq O(\epsilon^2)$, where θ^U is uniformly sampled from $\{\theta^0, \dots, \theta^{T-1}\}$. Furthermore, if the gradient domination condition holds for each agent (assumption D.3), then the converged point is an ϵ equilibrium.

$$\mathbb{E}[J(\theta^U)] \geq \sup_{\pi_j} \mathbb{E}[J(\pi_{\theta^U}^{-j}, \pi^j)] - O(\epsilon), \quad \forall j \in [N].$$

Remark 4.3. The term $\max_j \Gamma_j$ appears in both complexity bounds because the smoothness of the objective function $J(\theta)$ (the hardness of the optimisation problem) scales according to the dependence structure of the MDP. The standard PG estimates incur a complexity that scales with N^2 as it cannot exploit the local reward structure. On the other hand, the dependence-graph estimator improves this scale with Γ instead, which can be substantially smaller when inter-agent dependencies are sparse.

We compare our result with the state-of-the-art sample complexity of single-agent PG methods (Yuan et al., 2022) in the following two special cases.

Remark 4.4. As a first special case, when the dependence graph is totally disconnected (no edges exist between vertices of different agents, i.e. no cross-agents dependency or interactions), then our MDP formulation behaves as if a vector of parallel (and different) single-agent MDPs, where each "sample" in this vector MDP comprises N "subsample" tuples. In this case, we have that $\Gamma = O(N)$ and $\max_j \Gamma_j = O(1)$, and so the sample complexity of the dependence graph scales with N^2 vector samples, which becomes N^3 if we consider subsamples. As a result, if we consider a subsample in the vector samples equivalent to a sample in single-agent problems, then the sample complexity of our approach matches (up to a logarithmic factor) the sample complexity of solving N independent single-agent RL problems with the PG method to an equivalent $\epsilon/\sqrt{N}$ precision.

Remark 4.5. In the second special case, when the dependence graph is fully connected, we have that $\Gamma = O(N^2)$ and $\max_j \Gamma_j = O(N)$, and so the sample complexity of the dependence graph PG scales with N^4 . In this case, the sample complexity of our method matches the sample complexity of solving a single agent RL in the joint state action space (upto a constant factor due to the mismatch of the smoothness and Lipschitz coefficients in the joint and local parameter space) using PG method with the new $R_{\max}^{\text{new}} = NR_{\max}$.

Remark 4.4 and 4.5 demonstrate that our method provides a smooth transition between the two extreme settings of global and local learning depending on the sparsity of the agent interactions.

4.3 Dependence Graph Policy Gradient with approximate Graphs

The formulation of the dependence graph $\mathcal{G}$ introduced earlier assumes knowledge of the underlying transition dynamics $\mathbf{P}$. In practice, however, the transition model is often unknown, necessitating the use of approximate or heuristic graphs. In fact, prior works either assume that the graph is given (Qu et al., 2020), or use predefined structures (Ma et al., 2024; Jing et al., 2024). In this section, we consider how an approximate graph $\mathcal{G}'$ can be used in place of the true graph. We first establish a condition under which the estimate policy gradient remains accurate.

Lemma 4.6 (Gradient Error under an Approximated Dependence Graph). *Let $\mathcal{G}$ be the true dependence graph induced by the transition kernel $\mathbf{P}$, and let $\mathcal{G}'$ be a proper approximate graph for which there exists a transition kernel $\mathbf{P}'_{\mathcal{G}'}$ that admits $\mathcal{G}'$ as its dependence graph and satisfies*

$$\sup_{\mathbf{s}, \mathbf{a}} \|\mathbf{P}(\cdot | \mathbf{s}, \mathbf{a}) - \mathbf{P}'_{\mathcal{G}'}(\cdot | \mathbf{s}, \mathbf{a})\|_1 \leq \varepsilon. \quad (6)$$

Let $\nabla_{\pi_j} J(\pi; \mathcal{G}')$ denote the dependency-graph policy gradient from the graph $\mathcal{G}'$. If rewards are bounded in $[0, R_{\max}]$ and $B_j := \sup_{s^j, a^j} \|\nabla \log \pi_j(a^j | s^j)\|$ is finite, then

$$\|\nabla_{\pi_j} J^i(\pi) - \nabla_{\pi_j} J^i(\pi; \mathcal{G}')\| \leq O\left(B_j \frac{\gamma \varepsilon R_{\max}}{(1 - \gamma)^2}\right).$$

The subtlety of lemma 4.6 is the use of the approximated graph $\mathcal{G}'$ instead of $\mathcal{G}$ in the policy gradient while using samples from $\mathbf{P}$. The existence of the kernel $\mathbf{P}'_{\mathcal{G}'}$ is to only guarantee that $\mathcal{G}'$ represents something sensible but is not used otherwise. While Lemma 4.6 can be applied to any approximated graphs, including the heuristic ones, it also suggests a theoretical approach for estimating such a dependence graph through a model-based approach; by learning a dynamic model $\mathbf{P}'$, then the induced graph $\mathcal{G}'$ gives small-bias estimation of the gradient as long as the model prediction error is small. However, estimating the total variation in (6) of the world model can be challenging because it usually involves learning a high-dimensional, complex next-state distribution. Furthermore, it is unclear how to extract a dependence graph from a learned world model in practice. We next consider this quantity in a slightly different form.

For a given proper graph $\mathcal{G}'$, we let $\mathbf{P}_{\mathcal{G}'}$ be the probability of next states conditional on the graph structure of $\mathcal{G}'$, i.e.

$$P_{\mathcal{G}'}^i(s^{i'} | \mathbf{s}, \mathbf{a}) = P^i(s^{i'} | \{s^k, a^k : k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\}); \forall i \in [N].$$

When $\mathcal{G}'$ encompasses all edges in $\mathcal{G}$, then $\mathbf{P}_{\mathcal{G}'}$ recovers $\mathbf{P}$, otherwise, $\mathbf{P}_{\mathcal{G}'}$ is a marginal probability of $\mathbf{P}$ whose marginalization is edges present in $\mathcal{G}$ but not in $\mathcal{G}'$. By definition, $\mathbf{P}_{\mathcal{G}'}$ is a transition kernel that admits $\mathcal{G}'$ as its dependence graph (it is, in fact, the best approximated $\mathbf{P}'$ on $\mathcal{G}'$). Pinsker's inequality gives us an upper bound

$$\mathbb{E} \|P^i - P_{\mathcal{G}'}^i\|_1 \leq \sqrt{2} \sum_{j,n=1}^N \mathbb{E} \left[\sqrt{I(S^{i'}; (S^j, A^j))} \middle| \text{Pa}_n^i(s^i) \right] \quad (7)$$

where $\{\text{Pa}_n^i\}_{n=1}^N$ is a collection of non-decreasing agent-dependence sets for each s^i such that $\text{Pa}_1^i(s^i) = \text{Pa}_{\mathbf{P}}^i(s^i) \cap \text{Pa}_{\mathcal{G}'}^i(s^i)$, $\text{Pa}_N^i(s^i) = \text{Pa}_{\mathbf{P}}^i(s^i)$, and $|\text{Pa}_{m+1}^i(s^i) \setminus \text{Pa}_m^i(s^i)| \leq 1$. Furthermore, with some abuse of notations, we denote $\text{Pa}^i(\cdot)$ the states and actions of agents in $\text{Pa}^i(\cdot)$. Full derivation and other details of this result are left to Appendix C. Intuitively, if agent i 's states and actions have little impact on agent j 's future states, then the mutual information is low, and thus agent j can be considered not to be connected to agent i in the dependence graph. As a result, we can construct $\mathcal{G}'$ so that the upper bound in (7) is small, leading to small error gradients as in Lemma 4.6.

4.4 Approximating Dependence Graph via reverse world models

We conclude our method by proposing a simple approach for learning dependence graphs that circumvents learning a forward world model. To make the estimates in (7) practical, we introduce two simplifications to ease the mutual information estimation problem.

First, the mutual information in (7) can be written as $I(S^{i'}; S^j, A^j) = I(S^{i'}; S^j) + I(S^{i'}; A^j | S^j)$. As the MI between states $S^{i'}$ and S^j can be difficult to estimate, we will ignore the former term and assume that the next states only depend on the actions of other agents. Second, we approximate the states and actions in $\text{Pa}_n^i(s^i)$ defined in (7) with (s^i, a^i) . This approximation is accurate when the dependence graph has a low degree of vertices, such as 1 or 2. Overall, with these simplifications, our objective derived from (7) becomes

$$L_i(\pi) := \mathbb{E} \sum_j^N I(S^{i'}; A^j | S^i, A^i, S^j) = \mathbb{E} \sum_j^N [H(A^j | S^j) - H(A^j | S^i, S^j, S^{i'})]. \quad (8)$$

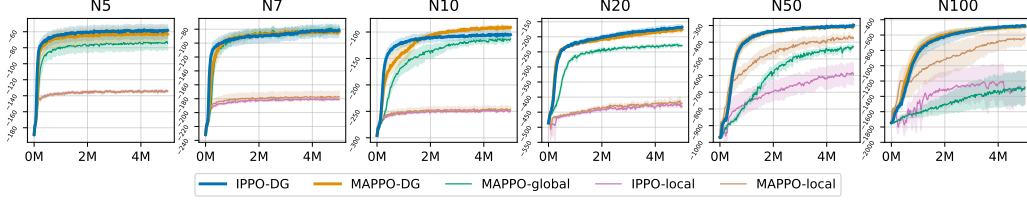


Figure 4: Results on the Spread MPE with known dependence graphs (DGs) and varying numbers of agents N . Shaded regions denote 95% stratified bootstrap confidence intervals. Global rewards perform well for small N but scale poorly as N increases, whereas local rewards scale better. Our method combines the advantages of both, maintaining strong performance across scales.

To estimate the entropies in (8), we adopt a variational lower bound based on a learned reverse dynamics model. Specifically, we train (i) an action prediction model q_ϕ to estimate $\hat{H}_{q_\phi}(A^j | S^j)$ and (ii) a multi-agent reverse world model q_ψ that predicts agent j 's action conditioned on the state information of both agents i and j . To determine whether a directed edge from agent j to agent i exists at state s , we compute the discrepancy between the two entropy terms. If conditioning on agent i significantly reduces the uncertainty of A^j , then we consider there is an edge between them. To construct $\mathcal{G}'$, we apply a soft thresholding as a heuristic rule to define an edge from j to i whenever

$$\frac{\hat{H}_{q_\psi}(A^j | s^i, s^j, s^{i'})}{\hat{H}_{q_\phi}(A^j | s^j)} < c,$$

where c is a hyperparameter in the range of $[0, 1]$. The value of c can be used to trade off the density of the approximated graph $\mathcal{G}'$ for accuracy. At the extreme value $c = 0$, our method reverts to the standard local reward setup. In our experiments, we manually select c at 0.9 for all experiments.

To improve the effectiveness of our approach, we additionally learn an encoder and incorporate the coordination graph into the GAE estimation, details and motivation designs are left to appendix B.4.

5 Experiments

We evaluate our methods on three MARL benchmarks, namely MPE, LBF, and SMAClite, which we modified to support local reward settings. Due to space constraints, we defer the environment details to Appendix E. Since our method is compatible with any on-policy gradient method, we adopt our backbone algorithms on both MAPPO Yu et al. (2022) and IPPO De Witt et al. (2020). Our source code is publicly available³.

Star-sread MPE. In the first part of the experiments, we evaluate our method under a known Dependence Graph (DG) setting. We design a simple MPE environment with a star-structured reward graph that is sufficiently sparse while still requiring coordination among agents. Figure 4 demonstrates the effectiveness of our approach, with IPPO-DG and MAPPO-DG achieving competitive performance compared to both local and global-reward baselines. Moreover, the performance gap between the DG-based methods and the global-reward baseline widens as the number of agents increases, which supports our theoretical analysis.

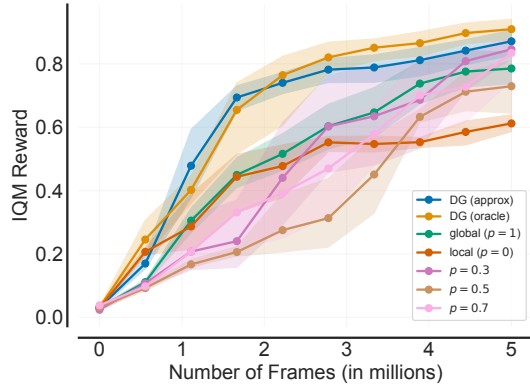


Figure 3: We test our methods (DG) with various random dependence graph structures, where each agent has a probability of p to form an edge with any other agent at any state. Global and local reward settings can be considered as special cases of random graphs with $p = 1$ and $p = 0$, respectively.

³<https://github.com/giangbang/dependence-graph-epymarl>



Figure 5: Results on the LBF benchmark on 6 selected scenarios following the evaluation protocol in Agarwal et al. (2021). Methods with (learned) dependence graphs consistently outperform other baselines from both local and global reward settings.

LBF with learned graphs. Figure 5 presents the results on the LBF benchmark. Our method improves performance for both IPPO and MAPPO, achieving the best results among all baselines. Specifically, using the Dependence Graph, IPPO increases its interquartile mean (IQM) reward to 0.89, compared to 0.66 with local rewards and 0.68 with global rewards. MAPPO shows a similar trend: IQM improves to 0.85 with the Dependence Graph, compared to 0.58 under local rewards and 0.77 under global rewards. Dependence Graph yields slightly larger gains for IPPO than for MAPPO due to its stronger performance under the local reward setup. Full experimental results are provided in Appendix E.2.

We conduct additional studies on MAPPO with different graph structures in LBF. As baselines, we choose random graph structures in which each agent has a probability p of connecting to the other agents. We tested with different values of p corresponding from sparse to dense graph connections. Furthermore, we also construct a (heuristic) oracle dependence graph setup based on domain knowledge of the LBF transitions. Specifically, we consider two agents connected if their L1 distances are less than or equal to 2. If there are multiple such agents, we only keep the two closest ones. Figure 3 demonstrates the effectiveness of our graph approximation, which performs almost comparably with the oracle graphs.

6 Limitations and Conclusions

Limitations. In environments where the oracle and heuristic graphs are available, such as in MPE and LBF, our methods outperform both local and global approaches. Our method with learned graph works almost as the oracle graph in LBF, but does not improve on SMAClite. This reflects the main limitations of our approach:

- Our entire work assumes that the graph depends on local states ($\text{Pa}(s)$ in section 2). In particular, the learned graph section was designed specifically that the graph can be inferred from the local states alone. What will happen when this assumption does not hold remains open. Unfortunately, this assumption can be difficult to hold in general (we discuss this in appendix B.3). Modeling such non-local information can require some form of communication or central mediators. However, given that our graph formulation is already more general than previous works (Bagnell and Ng, 2005; Qu and Li, 2019; Qu et al., 2020; Jing et al., 2024), we consider it reasonable to leave it to future work.
- Our learned graph procedure makes aggressive simplification on the state dependence. In particular, it may fail when transition dynamics depend more on agents’ states than on their actions. Nevertheless, to the best of our knowledge, our work is also the first to attempt to learn the dependence graph from data. Furthermore, the reverse model is only able to learn information about the action, and we believe that capturing the state information requires some form of forward world model. Again, we leave this direction to future work.

Conclusion. In this work, we present a MARL framework that leverages dependence graphs that capture dependencies between agents and their local states to filter irrelevant reward signals and mitigate credit assignment. We also propose a practical method that remains effective under noisy approximated graphs. Experiments show that our approach performs competitively across both local and global reward settings, offering a robust balance between the two approaches.

References

- Shalev-Shwartz, S.; Shammah, S.; Shashua, A. Safe, multi-agent, reinforcement learning for autonomous driving. *arXiv preprint arXiv:1610.03295* **2016**,
- Wang, J.; Xu, W.; Gu, Y.; Song, W.; Green, T. C. Multi-agent reinforcement learning for active voltage control on power distribution networks. *Advances in neural information processing systems* **2021**, *34*, 3271–3284.
- Cao, Y.; Yu, W.; Ren, W.; Chen, G. An overview of recent progress in the study of distributed multi-agent coordination. *IEEE Transactions on Industrial informatics* **2012**, *9*, 427–438.
- Yuan, L.; Zhang, Z.; Li, L.; Guan, C.; Yu, Y. A survey of progress on cooperative multi-agent reinforcement learning in open environment. *arXiv preprint arXiv:2312.01058* **2023**,
- Samvelyan, M.; Rashid, T.; De Witt, C. S.; Farquhar, G.; Nardelli, N.; Rudner, T. G.; Hung, C.-M.; Torr, P. H.; Foerster, J.; Whiteson, S. The starcraft multi-agent challenge. *arXiv preprint arXiv:1902.04043* **2019**,
- Lowe, R.; Wu, Y. I.; Tamar, A.; Harb, J.; Pieter Abbeel, O.; Mordatch, I. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems* **2017**, *30*.
- Yu, C.; Velu, A.; Vinitzky, E.; Gao, J.; Wang, Y.; Bayen, A.; Wu, Y. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems* **2022**, *35*, 24611–24624.
- Rashid, T.; Samvelyan, M.; De Witt, C. S.; Farquhar, G.; Foerster, J.; Whiteson, S. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research* **2020**, *21*, 1–51.
- Chang, Y.-H.; Ho, T.; Kaelbling, L. All learning is local: Multi-agent learning in global reward games. *Advances in neural information processing systems* **2003**, *16*.
- Foerster, J.; Farquhar, G.; Afouras, T.; Nardelli, N.; Whiteson, S. Counterfactual multi-agent policy gradients. *Proceedings of the AAAI conference on artificial intelligence*. 2018.
- De Witt, C. S.; Gupta, T.; Makoviichuk, D.; Makovychuk, V.; Torr, P. H.; Sun, M.; Whiteson, S. Is independent learning all you need in the starcraft multi-agent challenge? *arXiv preprint arXiv:2011.09533* **2020**,
- Tan, M. Multi-agent reinforcement learning: Independent vs. cooperative agents. *Proceedings of the tenth international conference on machine learning*. 1993; pp 330–337.
- Zheng, L.; Yang, J.; Cai, H.; Zhou, M.; Zhang, W.; Wang, J.; Yu, Y. Magent: A many-agent reinforcement learning platform for artificial collective intelligence. *Proceedings of the AAAI conference on artificial intelligence*. 2018.
- Devlin, S.; Yliniemi, L.; Kudenko, D.; Tumer, K. Potential-based difference rewards for multiagent reinforcement learning. *Proceedings of the 2014 international conference on Autonomous agents and multi-agent systems*. 2014; pp 165–172.
- Le, B. G.; Ta, V. C. Toward finding strong pareto optimal policies in multi-agent reinforcement learning. *Machine Learning* **2025**, *114*, 1–20.
- Sunehag, P.; Lever, G.; Gruslys, A.; Czarnecki, W. M.; Zambaldi, V.; Jaderberg, M.; Lanctot, M.; Sonnerat, N.; Leibo, J. Z.; Tuyls, K.; others Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296* **2017**,
- Qu, G.; Lin, Y.; Wierman, A.; Li, N. Scalable multi-agent reinforcement learning for networked systems with average reward. *Advances in Neural Information Processing Systems* **2020**, *33*, 2074–2086.
- Bagnell, D.; Ng, A. On local rewards and scaling distributed reinforcement learning. *Advances in Neural Information Processing Systems* **2005**, *18*.

- Qu, G.; Li, N. Exploiting fast decaying and locality in multi-agent mdp with tree dependence structure. 2019 IEEE 58th conference on decision and control (CDC). 2019; pp 6479–6486.
- Jing, G.; Bai, H.; George, J.; Chakraborty, A.; Sharma, P. K. Distributed multiagent reinforcement learning based on graph-induced local value functions. *IEEE Transactions on Automatic Control* **2024**, *69*, 6636–6651.
- Christianos, F.; Schäfer, L.; Albrecht, S. Shared experience actor-critic for multi-agent reinforcement learning. *Advances in neural information processing systems* **2020**, *33*, 10707–10717.
- Kuba, J. G.; Wen, M.; Meng, L.; Zhang, H.; Mguni, D.; Wang, J.; Yang, Y.; others Settling the variance of multi-agent policy gradients. *Advances in Neural Information Processing Systems* **2021**, *34*, 13458–13470.
- Sun, M.; Devlin, S.; Beck, J.; Hofmann, K.; Whiteson, S. Trust region bounds for decentralized ppo under non-stationarity. *arXiv preprint arXiv:2202.00082* **2022**,
- Zhong, Y.; Kuba, J. G.; Feng, X.; Hu, S.; Ji, J.; Yang, Y. Heterogeneous-agent reinforcement learning. *Journal of Machine Learning Research* **2024**, *25*, 1.
- Wu, Z.; Yu, C.; Ye, D.; Zhang, J.; Zhuo, H. H.; others Coordinated proximal policy optimization. *Advances in Neural Information Processing Systems* **2021**, *34*, 26437–26448.
- Mao, H.; Gong, Z.; Xiao, Z. Reward design in cooperative multi-agent reinforcement learning for packet routing. *arXiv preprint arXiv:2003.03433* **2020**,
- Syed, S. Q.; Bai, H. Structured Cooperative Multi-Agent Reinforcement Learning Control: a Bayesian Network Perspective. *IEEE Transactions on Control of Network Systems* **2026**,
- Baxter, J.; Bartlett, P. L. Infinite-horizon policy-gradient estimation. *journal of artificial intelligence research* **2001**, *15*, 319–350.
- Yuan, R.; Gower, R. M.; Lazaric, A. A general sample complexity analysis of vanilla policy gradient. International Conference on Artificial Intelligence and Statistics. 2022; pp 3332–3380.
- Ma, C.; Li, A.; Du, Y.; Dong, H.; Yang, Y. Efficient and scalable reinforcement learning for large-scale network control. *Nature Machine Intelligence* **2024**, *6*, 1006–1020.
- Agarwal, R.; Schwarzer, M.; Castro, P. S.; Courville, A. C.; Bellemare, M. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems* **2021**, *34*, 29304–29320.
- Sutton, R. S.; Barto, A. G. *Reinforcement learning: An introduction*; MIT press, 2018.
- Pan, A.; Bhatia, K.; Steinhardt, J. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544* **2022**,
- Oroojlooy, A.; Hajinezhad, D. A review of cooperative multi-agent deep reinforcement learning. *Applied Intelligence* **2023**, *53*, 13677–13722.
- Hernandez-Leal, P.; Kaisers, M.; Baarslag, T.; De Cote, E. M. A survey of learning in multiagent environments: Dealing with non-stationarity. *arXiv preprint arXiv:1707.09183* **2017**,
- Leibo, J. Z.; Zambaldi, V.; Lanctot, M.; Marecki, J.; Graepel, T. Multi-agent reinforcement learning in sequential social dilemmas. *arXiv preprint arXiv:1702.03037* **2017**,
- Tampuu, A.; Matiisen, T.; Kodelja, D.; Kuzovkin, I.; Korjus, K.; Aru, J.; Aru, J.; Vicente, R. Multiagent cooperation and competition with deep reinforcement learning. *PloS one* **2017**, *12*, e0172395.
- Wang, L.; Yang, Z.; Wang, Z. Breaking the curse of many agents: Provable mean embedding Q-iteration for mean-field reinforcement learning. International conference on machine learning. 2020; pp 10092–10103.

- Wang, L.; Zhang, Y.; Hu, Y.; Wang, W.; Zhang, C.; Gao, Y.; Hao, J.; Lv, T.; Fan, C. Individual reward assisted multi-agent reinforcement learning. *International conference on machine learning*. 2022; pp 23417–23432.
- Gronauer, S.; Diepold, K. Multi-agent deep reinforcement learning: a survey. *Artificial Intelligence Review* **2022**, *55*, 895–943.
- Zhou, M.; Liu, Z.; Sui, P.; Li, Y.; Chung, Y. Y. Learning implicit credit assignment for cooperative multi-agent reinforcement learning. *Advances in neural information processing systems* **2020**, *33*, 11853–11864.
- Hong, Y.; Jin, Y.; Tang, Y. Rethinking individual global max in cooperative multi-agent reinforcement learning. *Advances in neural information processing systems* **2022**, *35*, 32438–32449.
- Wang, J.; Ren, Z.; Han, B.; Ye, J.; Zhang, C. Towards understanding linear value decomposition in cooperative multi-agent q-learning. **2020**,
- Rashid, T.; Farquhar, G.; Peng, B.; Whiteson, S. Weighted qmix: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning. *Advances in neural information processing systems* **2020**, *33*, 10199–10210.
- Guestrin, C.; Koller, D.; Parr, R. Multiagent planning with factored MDPs. *Advances in neural information processing systems* **2001**, *14*.
- Kok, J. R.; Vlassis, N. Collaborative multiagent reinforcement learning by payoff propagation. *Journal of machine learning research* **2006**, *7*.
- Böhmer, W.; Kurin, V.; Whiteson, S. Deep coordination graphs. *International Conference on Machine Learning*. 2020; pp 980–991.
- Weaver, L.; Tao, N. The optimal reward baseline for gradient-based reinforcement learning. *arXiv preprint arXiv:1301.2315* **2013**,
- Wu, C.; Rajeswaran, A.; Duan, Y.; Kumar, V.; Bayen, A. M.; Kakade, S.; Mordatch, I.; Abbeel, P. Variance reduction for policy gradient with action-dependent factorized baselines. *arXiv preprint arXiv:1803.07246* **2018**,
- Wang, J.; Zhang, Y.; Kim, T.-K.; Gu, Y. Shapley Q-value: A local reward approach to solve global reward games. *Proceedings of the AAAI conference on artificial intelligence*. 2020; pp 7285–7292.
- Lin, Y.; Qu, G.; Huang, L.; Wierman, A. Multi-agent reinforcement learning in stochastic networked systems. *Advances in neural information processing systems* **2021**, *34*, 7825–7837.
- McAllester, D.; Stratos, K. Formal limitations on the measurement of mutual information. *International Conference on Artificial Intelligence and Statistics*. 2020; pp 875–884.
- Agarwal, A.; Jiang, N.; Kakade, S. M.; Sun, W. Reinforcement learning: Theory and algorithms. *CS Dept., UW Seattle, Seattle, WA, USA, Tech. Rep* **2019**, *32*, 96.
- Michalski, A.; Christianos, F.; Albrecht, S. V. Smaclite: A lightweight environment for multi-agent reinforcement learning. *arXiv preprint arXiv:2305.05566* **2023**,
- Wang, J.; Ren, Z.; Liu, T.; Yu, Y.; Zhang, C. Qplex: Duplex dueling multi-agent q-learning. *arXiv preprint arXiv:2008.01062* **2020**,
- Papoudakis, G.; Christianos, F.; Schäfer, L.; Albrecht, S. V. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *arXiv preprint arXiv:2006.07869* **2020**,
- Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.; Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438* **2015**,
- Hu, J.; Jiang, S.; Harding, S. A.; Wu, H.; Liao, S.-w. Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning. *arXiv preprint arXiv:2102.03479* **2021**,
- Ba, J. L.; Kiros, J. R.; Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450* **2016**,

Appendix

A Related Work

Reward for Cooperation. The behaviors of RL agents are defined by the reward structure, a principle known as the reward hypothesis (Sutton and Barto, 2018). Effective reward functions are notoriously difficult to design, and this remains true in cooperative MARL (Pan et al., 2022). Traditionally, agents are trained using separate, agent-specific reward vectors under the independent learning framework, where each agent is treated as an individual RL problem (Tan, 1993; De Witt et al., 2020). This approach offers benefits such as simplicity and scalability (Oroojlooy and Hajinezhad, 2023), but also suffers from issues like non-stationarity and miscoordination (Hernandez-Leal et al., 2017). The degree of cooperation is also sensitive to reward shapings (Leibo et al., 2017; Tampuu et al., 2017).

In contrast, using a shared reward function directly enforces cooperation as in fully cooperative MARL settings (Yuan et al., 2023). This setup is often trained under the Centralized Training with Decentralized Execution (CTDE) paradigm (Foerster et al., 2018), which further enhances coordination through centralized training. However, shared rewards introduce the credit assignment problem, making it difficult to scale fully cooperative learning to environments with many agents (Bagnell and Ng, 2005; Wang et al., 2020). As a result, local reward learning is more suitable for large-scale MARL problems (Zheng et al., 2018).

Recently, a number of approaches attempt to explore a middle ground between the two aforementioned reward settings. For instance, Le and Ta (2025) apply tools from multi-objective optimization to identify Pareto optimal policies. However, their method treats the problem as a generic optimization task and does not explicitly model individual agent contributions within the sequential decision-making process. Wang et al. (2022) optimize auxiliary local rewards alongside the sparse global team reward. However, ultimately this approach still solves the two different reward formulations and relies on additional heuristic design of local rewards to enable effective learning.

Credit assignment. Credit assignment problem is the problem of attributing the contribution of individual agents from the team’s rewards (Chang et al., 2003). Two main approaches to solving credit assignment problems in the literature include (i) decomposing the global value function, and (ii) marginalizing the contribution of one agent over the others (Gronauer and Diepold, 2022). The first approach, sometimes referred to as implicit credit assignment (Zhou et al., 2020), has gained popularity from the works of VDN (Sunehag et al., 2017) and QMIX (Rashid et al., 2020). This approach is characterized by using individual utility functions and a mixing procedure to combine them into a total joint value function. To facilitate efficient decentralized execution, most methods follow the Individual-Global-Max (IGM) principle (Hong et al., 2022). IGM ensures local greedy agents align with globally optimal actions, but in practice, its limited decomposability can hinder convergence (Wang et al., 2020; Rashid et al., 2020). Coordination Graph (Guestrin et al., 2001) provides another approach that factorizes the joint value function using a graph structure, enabling a higher-order value factorizations (Kok and Vlassis, 2006). However, this method introduces significant computational overhead, which in practice often restricts implementations to pairwise interactions (Böhmer et al., 2020). The second approach, with representative works such as Counterfactual learning (Foerster et al., 2018), mainly uses baselines (Weaver and Tao, 2013; Wu et al., 2018) to reduce the variance of Policy Gradient estimation (Kuba et al., 2021). Shapley Q-value (Wang et al., 2020) is a similar approach that assigns credit based on agents’ average marginal contribution over all coalition formations. A common characteristic of these methods is that they start from the global reward and distribute credit in a top-down manner. In contrast, our work avoids the credit assignment problem by directly using local rewards in a bottom-up approach.

Networked Multi-Agent MDPs. Theoretically, our formulation is related to the works that model agent interactions as a networked graph in the Multi-Agent MDP framework (Bagnell and Ng, 2005; Qu and Li, 2019; Qu et al., 2020). This formulation enables more fine-grained modeling of local interactions, which is useful for analyzing agents’ contributions. Most works in this direction rely on the exponential decay property of value representations (Qu et al., 2020) to approximate the value function using a small subset of agents, possibly with communications (Ma et al., 2024), while our work explores the time dimension of value estimation along the dependency graph paths. Recent works (Jing et al., 2024; Syed and Bai, 2026) further show that the policy gradient depends only on the reachable subset of agents, which corresponds to the infinite meeting-time case in our framework. Another related direction is explored in (Lin et al., 2021), which generalizes the setting to stochastic

graphs. While our work assumes deterministic graphs, the dependence paths on the graphs evolve with the states visited over time and can therefore be regarded as effectively stochastic due to the stochasticity of future trajectories.

B Further details omitted in the main text

B.1 Multi-agent Policy Gradient

The extension of the policy gradient theorem to multi-agent learning is introduced and proved in (Foerster et al., 2018), which we reproduce here for completeness. From the single-agent policy gradient theorem, we have

$$\begin{aligned}\nabla_{\pi_i} J(\pi) &= \mathbb{E}[\nabla_{\pi_i} \log \pi(\mathbf{a}|\mathbf{s})Q(\mathbf{s}, \mathbf{a})] \\ &= \mathbb{E}\left[\nabla_{\pi_i} \log \prod \pi_i(a^i|s^i)Q(\mathbf{s}, \mathbf{a})\right] \\ &= \mathbb{E}\left[\sum_i^N \nabla \log \pi_i(a^i|s^i)Q(\mathbf{s}, \mathbf{a})\right]\end{aligned}$$

where the second equality is based on the independent policy assumption. With weight sharing, the global gradient equals the sum of the gradients of individual agents. In case that we want to write the gradient of one agent to another, as in Eq. (2), we can set $r = r^i$; the global reward as the reward of agent i , and using the multi-agent policy gradient theorem as above.

B.2 Discussion and limitations

Compare with the fully cooperative setting. While our approach requires the additional local-reward information, which may not be available for all environments. It is common that the reward from the scalar reward team can be attributed to a specific agent. For example, in the LBF benchmark, the reward corresponds to agents participating in the food foraging. Even when the reward cannot be attributed explicitly, we can always distribute it evenly among the team, which essentially reduces our setting to a fully cooperative one. Additionally, scalarizing the rewards is a one-way operator, where any vector-valued reward can be transformed into a scalar team reward, but the converse is not possible. Furthermore, even when the individual reward signals are available, existing methods do not provide principled approaches to leverage this reward vector to mitigate the credit assignment problems. In contrast, our work offers a theory-grounded framework for incorporating individual reward information directly into the optimization of the team objective.

Limitations. Our approach to graph approximation depends on the assumption that the neighboring agents can be inferred from the local observation, which technically is difficult to satisfy in practice (see Appendix B.3 for a discussion). Furthermore, the effect of ignoring the state mutual information in Section 4.4 is unknown, and we expect it to fail in environments where the transition probabilities depend heavily on agents’ states instead of actions.

B.3 Factorization of states in practice

Our theoretical ground for the graph approximation procedure is based on the assumption that an agent can infer their neighbor agents from observed states, as introduced in Section 2. This assumption naturally allows for the graph approximation idea based on local world model learning. In essence, the decomposition graph requires that current substates contain enough information to determine which agents it can interact with that can influence its next states. Strictly speaking, this requirement can be difficult to meet in practice. We demonstrate this argument in an illustrative example in this section. More specifically, we consider a simplified environment in Figure 6. In this example, the states of each agent are the field of vision where it can observe which other agents are adjacent in their

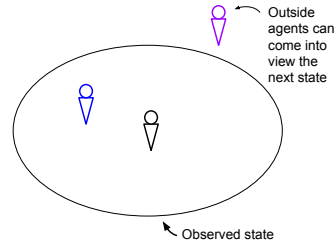


Figure 6: A simplified example, the purple agent can go into the observed states of the black agent and cannot be captured by a dependency graph based on the current substates of the black agent.

vicinity (the oval circle). Next states of one agent not only depend on the nearby agents, but can also depend on the outside agents that can jump into its vision. A decomposition graph that is built on this observation state alone can not fully capture such "outsider" influences. However, we can consider that their overall effect can be negligible if the events of outside agents' interference happen rarely.

B.4 Practical Implementation

To make our method more effective in practice, two additional challenges must be addressed. First, the approximated graph on the original state space may contain many spurious edges. For example, agents may partially observe other agents' local states, even when it does not have a direct impact on their future states; such observations can allow for more accurate prediction of other agents' actions, resulting in many redundant edges. Second, the approximated entropy difference in (8) is neither a lower nor an upper bound of the mutual information, as discussed in McAllester and Stratos (2020), which can lead to noisy graph approximations.

To mitigate the first issue, we adopt a learned latent state that filters out irrelevant information. Specifically, we train an encoder E that maps the original states to a latent space that captures the information relevant only to the current agent by using a single-agent reverse world model q_φ . This single-agent reverse world model guides the encoder to retain only the information that is necessary for the current agent while ignoring the extraneous information about others. Our overall model approximation scheme with reverse world models is illustrated in Figure 7.

To address the second issue, we devise an estimate strategy that combines multiple meeting timesteps t' . In particular, Proposition 4.1 shows that the true meeting timestep t' provides the tightest estimate; however, any other estimate using a timestep earlier than t' will also have the same expected gradient. As a result, we compute an exponentially weighted average of the dependence-graph gradients over all meeting timesteps earlier than the approximated timestep t' . This aggregation scheme is analogous to the GAE scheme, which results in a novel GAE-like estimate that incorporates the dependence graphs. The overall procedure is presented in Algorithm 1.

B.5 Multi-Agent GAE- λ with dependence graph

The key idea behind GAE is to average over all the n -step return estimations, which balances the high variance of Monte Carlo sampling and the high bias of the short-horizon n -step returns. We employ the same ideas as GAE for the meeting timestep t' , where we do not exclusively rely on a single estimate of this quantity. Similar to GAE, we start from the definition of TD error, defined as $\delta_t = r_t + \gamma V^i(s_{t+1}) - V^i(s_t)$, where $V^i(\cdot)$ are value estimates of the agent i 's policy. The multi-step advantage estimates from δ , combined with the dependence graphs at each timestep, are then defined as follows, for a given trajectory τ ,

$$\begin{aligned}\hat{A}_t^{(1)} &= \delta_t \\ \hat{A}_t^{(2)} &= 1(j \in \text{Pa}^i(s_t^i, 1, \tau))\delta_t + \gamma\delta_{t+1} \\ &\vdots \\ \hat{A}_t^{(k+1)} &= \sum_{l=0}^{k-1} \gamma^l 1(j \in \text{Pa}^i(s_{t+l}^i, l, \tau))\delta_{t+l} + \gamma^k \delta_{t+k},\end{aligned}$$

where $\text{Pa}^i(s_t, t, \tau)$ is the t -hop parents of s_t^i , defined in (10). Each of the $\hat{A}_t^{(k)}$ involves the sum of k TD error terms. Furthermore, all $\hat{A}_t^{(k)}$ are equivalent to advantage estimation with meeting times k upto $t' > k$, with t' the predicted first timestep that i and j are connected by a path. This can be

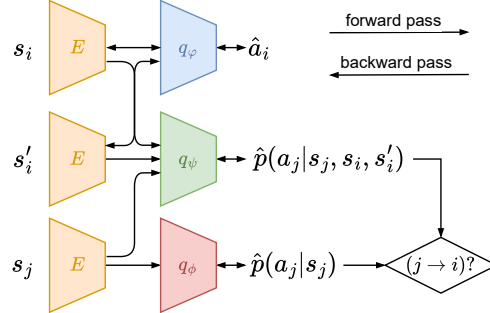


Figure 7: Dependence graph approximation via reverse world models.

effective for mitigating the noisy predictions of the dependence graphs inferred by the world models. Similar to GAE, we take the exponentially weighted average of all these terms

$$\begin{aligned}
\hat{A}_t^{\text{GAE}} &= (1 - \lambda) \left(\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots \right) \\
&= (1 - \lambda) \left(\delta_t + \lambda (1(j \in \text{Pa}^i(\mathbf{s}_t^i, t, \tau)) \delta_t + \gamma \delta_{t+1}) \right. \\
&\quad \left. + \lambda^2 (1(j \in \text{Pa}^i(\mathbf{s}_t^i, t, \tau)) \delta_t + \gamma 1(j \in \text{Pa}^i(\mathbf{s}_{t+1}^i, t+1, \tau)) \delta_{t+1} + \gamma^2 \delta_{t+2}) + \dots \right) \\
&= (1 - \lambda) \left(\delta_t (1 + 1(j \in \text{Pa}^i(\mathbf{s}_t^i, t, \tau)) (\lambda + \lambda^2 + \dots)) \right. \\
&\quad \left. + \gamma \delta_{t+1} (\lambda + 1(j \in \text{Pa}^i(\mathbf{s}_{t+1}^i, t+1, \tau)) (\lambda^2 + \lambda^3 + \dots)) + \dots \right) \\
&= \lambda (1 - \lambda) \left(\delta_t 1(j \in \text{Pa}^i(\mathbf{s}_t^i, t, \tau)) (1 + \lambda + \lambda^2 + \dots) \right. \\
&\quad \left. + \gamma \delta_{t+1} 1(j \in \text{Pa}^i(\mathbf{s}_{t+1}^i, t+1, \tau)) (\lambda + \lambda^2 + \dots) + \dots \right) \\
&\quad + (1 - \lambda) (\delta_t + \gamma \lambda \delta_{t+1} + \dots) \\
&= \sum_{l=0}^{\infty} \gamma^l \lambda^{l+1} \delta_{t+l} 1(j \in \text{Pa}^i(\mathbf{s}_{t+l}^i, t+l, \tau)) + (1 - \lambda) \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \\
&= \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \left(\lambda 1(j \in \text{Pa}^i(\mathbf{s}_{t+l}^i, t+l, \tau)) + 1 - \lambda \right).
\end{aligned}$$

Intuitively, when two agents are not connected by a path, we do not discard the contribution of the other agent’s gradient entirely; instead, we downweight it by a factor of $1 - \lambda$. Finally, to find the t -hop parent set, we perform successive matrix multiplications of the adjacency matrices to the current timestep t as detailed in Algorithm 1.

Algorithm 1 Policy Gradient with Multi-Agent GAE- λ and Dependence Graph

input Trajectory $\tau = \{(\mathbf{s}_t, r_t^i)\}_{t=0}^{T-1}$, value estimates $\{V^i(\mathbf{s}_t)\}_{t=0}^T$, discount factor γ , GAE parameter λ , adjacency matrix $\{\mathbf{A}^t\}_{t=0}^{T-1}$, action scores $\{\nabla \log(\pi_j(a_t^j | \mathbf{s}_t^j))\}_{t=0}^{T-1}$

output Gradient $\hat{\nabla}_{\pi_j} \mathcal{J}^i(\pi; \mathcal{G})$, Advantage $\{A_t^i\}_{t=0}^{T-1}$

- 1: Initialize $A_t \leftarrow 0$ for all $t = 0, \dots, T-1$
- 2: **for** $t_0 = T-1$ **down to** 0 **do**
- 3: $\text{gae} \leftarrow 0$
- 4: $\mathbf{H} \leftarrow \text{diag}(\mathbf{1})$
- 5: **for** $t = t_0$ **to** $T-1$ **do**
- 6: $\delta_t^i \leftarrow r_t^i + \gamma V^i(\mathbf{s}_{t+1}) - V^i(\mathbf{s}_t)$
- 7: $\mathbf{H} \leftarrow \mathbf{H} \cdot \mathbf{A}^t$
- 8: $\text{gae} \leftarrow \delta_t^i (\gamma \lambda)^{t-t_0} (\lambda 1(\mathbf{H}_{ji} > 0) + 1 - \lambda) + \text{gae}$
- 9: **end for**
- 10: $A_{t_0}^i \leftarrow \text{gae}$
- 11: **end for**

return $\frac{1}{T} \sum_{t=0}^{T-1} \gamma^t A_t^i \nabla \log \pi_j(a_t^j | \mathbf{s}_t^j)$, $\{A_t^i\}_{t=0}^{T-1}$

C Proofs for section 4.1

In this section, we use the following definition of path on the dependence graphs.

Definition C.1 (Path in the Dependence Graph). Let $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$ be the induced Dependence Graph of $\mathcal{M}$. A *path* from vertex $(\mathbf{s}, i)$ to vertex $(\mathbf{s}', j)$ is a sequence of vertices

$$(\mathbf{s}_0, i_0) \rightarrow (\mathbf{s}_1, i_1) \rightarrow \dots \rightarrow (\mathbf{s}_L, i_L),$$

such that

1. $i_0 = i, i_L = j, \mathbf{s}_0 = \mathbf{s}, \mathbf{s}_L = \mathbf{s}'$,
2. for each $\ell = 0, \dots, L-1$, the directed edge $(\mathbf{s}_\ell, i_\ell) \rightarrow (\mathbf{s}_{\ell+1}, i_{\ell+1})$ belongs to $\mathcal{E}$.

We call L the *length* of the path. If there is a path from $(\mathbf{s}_t, i)$ to $(\mathbf{s}_{t'}, j)$, then there is a path from $(\mathbf{s}_t, i)$ to any of $(\mathbf{s}_{t''}, j)$ for all $t'' \geq t'$ since an agent influences its next states. A path is defined on a realizable sequence of states that we call a (state) trajectory.

Proof of Proposition 4.1. From the policy gradient theorem, we unroll each timestep

$$\begin{aligned}
(1 - \gamma) \nabla_{\pi_j} J^i(\boldsymbol{\pi}) &= \mathbb{E}_{\mathbf{s}_0 \sim \rho_{\boldsymbol{\pi}}, \mathbf{a}_0 \sim \boldsymbol{\pi}} [\nabla \log \pi_j(a_0^j | s_0^j) Q^i(\mathbf{s}_0, \mathbf{a}_0)] \\
&= \mathbb{E}_{\rho, \boldsymbol{\pi}} \left[\nabla \log \pi_j(a_0^j | s_0^j) \left[\mathbb{E}[r_0^i(s_0^i, a_0^i)] + \gamma \mathbb{E}_{\mathbf{s}_1, \boldsymbol{\pi}} [Q^i(\mathbf{s}_1, \mathbf{a}_1) | \mathbf{s}_0, \mathbf{a}_0] \right] \right] \\
&= \mathbb{E}_{\rho, \boldsymbol{\pi}} \left[\nabla \log \pi_j(a_0^j | s_0^j) \gamma \mathbb{E}_{\mathbf{s}_1, \boldsymbol{\pi}} [Q^i(\mathbf{s}_1, \mathbf{a}_1) | \mathbf{s}_0, \mathbf{a}_0] \right] \\
&= \mathbb{E}_{\rho, \boldsymbol{\pi}} \left[\nabla \log \pi_j(a_0^j | s_0^j) \left[\gamma \mathbb{E}_{\mathbf{s}_1^i, \mathbf{a}_1^i} [r_1^i(s_1^i, a_1^i) | \mathbf{s}_0, \mathbf{a}_0] + \gamma^2 \mathbb{E}_{\mathbf{s}_2, \mathbf{a}_2} [Q^i(\mathbf{s}_2, \mathbf{a}_2) | \mathbf{s}_0, \mathbf{a}_0] \right] \right], \tag{9}
\end{aligned}$$

where in the third equality, we use $\mathbb{E}_{a_0^j} [\nabla \log \pi_j(a_0^j | s_0^j) r_0^i(s_0^i, a_0^i)] = 0$ when $i \neq j$ (since we are assuming local reward structure), which we can write as

$$(1 - \gamma) \nabla_{\pi_j} J^i(\boldsymbol{\pi}) = \mathbb{E} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_\tau [r_0^i 1(j \in \{i\}) + \gamma Q^i(\mathbf{s}_1, \mathbf{a}_1) | \mathbf{s}_0, \mathbf{a}_0] \right].$$

Next, observe the first term in (9) that

$$\mathbb{E}_{\mathbf{s}_1^i, \mathbf{a}_1^i} [r_1^i(s_1^i, a_1^i) | \mathbf{s}_0, \mathbf{a}_0] = \sum_{s_1^i \in \mathcal{S}^i} \sum_{a_1^i \in \mathcal{A}^i} P^i(s_1^i | \{s_0^k, a_0^k : k \in \text{Pa}^i(s_0^i)\}) \pi_i(a_1^i | s_1^i) r_1^i(s_1^i, a_1^i)$$

So given $\mathbf{s}_0$, if $j \notin \text{Pa}^i(s_0^i)$, then

$$\begin{aligned}
&\mathbb{E}_{\mathbf{a}_0 \sim \boldsymbol{\pi}} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{\mathbf{s}_1^i, \mathbf{a}_1^i} [r_1^i(s_1^i, a_1^i) | \mathbf{a}_0] \mid \mathbf{s}_0 \right] \\
&= \mathbb{E}_{\mathbf{a}_0} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{\mathbf{s}_1^i, \mathbf{a}_1^i} [r_1^i(s_1^i, a_1^i) | \mathbf{a}_0^{-j}] \mid \mathbf{s}_0 \right] \\
&= 0.
\end{aligned}$$

Note that we can interpret the property $j \notin \text{Pa}^i(s_0^i)$ as: given a segment of state trajectory $\mathbf{s}_{0:1} = (\mathbf{s}_0, \mathbf{s}_1)$ for an arbitrary $\mathbf{s}_1 \in \mathcal{S}$, then the path (defined on this segment of trajectory) $(\mathbf{s}_0, j) \rightarrow (\mathbf{s}_1, i)$ is not a path in the dependency graph. If this property holds, then the policy gradient of agent j with respect to the reward of agent i at the current timestep $t = 1$ is 0. In other words,

$$(1 - \gamma) \nabla_{\pi_j} J^i(\boldsymbol{\pi}) = \mathbb{E} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_\tau [r_0^i 1(j \in \{i\}) + \gamma r_1^i 1(j \in \text{Pa}^i(s_0^i)) + \gamma^2 Q^i(\mathbf{s}_2, \mathbf{a}_2) | \mathbf{s}_0, \mathbf{a}_0] \right].$$

We next consider the second term in (9)

$$\begin{aligned}
&\mathbb{E}_{\rho, \boldsymbol{\pi}} \left[\nabla \log \pi_j(a_0^j | s_0^j) \left[\gamma^2 \mathbb{E}_{\mathbf{s}_2, \mathbf{a}_2} [Q^i(\mathbf{s}_2, \mathbf{a}_2) | \mathbf{s}_0, \mathbf{a}_0] \right] \right] \\
&= \mathbb{E}_{\rho, \boldsymbol{\pi}} \left[\nabla \log \pi_j(a_0^j | s_0^j) \left[\gamma^2 \mathbb{E}_{\mathbf{s}_2^i, \mathbf{a}_2^i} [r_2^i(s_2^i, a_2^i) | \mathbf{s}_0, \mathbf{a}_0] + \gamma^3 \mathbb{E}_{\mathbf{s}_3, \mathbf{a}_3} [Q^i(\mathbf{s}_3, \mathbf{a}_3) | \mathbf{s}_0, \mathbf{a}_0] \right] \right].
\end{aligned}$$

Again, we write the reward term in its explicit form

$$\begin{aligned}
& \mathbb{E}_{s_2^i, a_2^i} [r^i(s_2^i, a_2^i) | \mathbf{s}_0, \mathbf{a}_0] \\
&= \sum_{\mathbf{s}_1 \in \mathcal{S}} \sum_{\mathbf{a}_1 \in \mathcal{A}} \sum_{s_2^i \in \mathcal{S}^i} \sum_{a_2^i \in \mathcal{A}^i} \left[r_2^i(s_2^i, a_2^i) P^i(s_2^i | \{s_1^k, a_1^k : k \in \text{Pa}^i(s_1^i)\}) \pi_i(a_2^i | s_2^i) \right. \\
&\quad \left. \prod_n^N \left[P^n(s_1^n | \{s_0^m, a_0^m : m \in \text{Pa}^n(s_0^n)\}) \pi_n(a_1^n | s_1^n) \right] \right] \\
&= \sum_{\mathbf{s}_1 \in \mathcal{S}} \sum_{\mathbf{a}_1 \in \mathcal{A}} \sum_{s_2^i \in \mathcal{S}^i} \sum_{a_2^i \in \mathcal{A}^i} \left[r_2^i(s_2^i, a_2^i) P^i(s_2^i | \{s_1^k, a_1^k : k \in \text{Pa}^i(s_1^i)\}) \pi_i(a_2^i | s_2^i) \right. \\
&\quad \left. \prod_{k \in \text{Pa}^i(s_1^i)} \left[P^k(s_1^k | \{s_0^m, a_0^m : m \in \text{Pa}^k(s_0^k)\}) \pi_k(a_1^k | s_1^k) \right] \right],
\end{aligned}$$

the second equality is due to the marginal expectation of agents that are not in $\text{Pa}^i(s_1^i)$. So given $\mathbf{s}_0$ and $\mathbf{s}_1$, if $j \notin \cup_{k \in \text{Pa}^i(s_1^i)} \text{Pa}^k(s_0^k)$, then

$$\mathbb{E}_{\mathbf{a}_0 \sim \pi} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{s_2^i, a_2^i} [r(s_2^i, a_2^i) | \mathbf{a}_0, \mathbf{s}_1] \mid \mathbf{s}_0 \right] = 0,$$

we can interpret the property $j \notin \cup_{k \in \text{Pa}^i(s_1^i)} \text{Pa}^k(s_0^k)$ as: given a segment of trajectory $\mathbf{s}_{0:2} = (\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2)$ for an arbitrary $\mathbf{s}_2 \in \mathcal{S}$, then all the paths (defined on this segment of trajectory) $(\mathbf{s}_0, j) \rightarrow (\mathbf{s}_1, \cdot) \rightarrow (\mathbf{s}_2, i)$ is not a path in the dependency graph $\mathcal{G}$. If this property holds, then the policy gradient of agent j with respect to the reward of agent i at the current timestep $t = 2$ is 0. In other words,

$$\begin{aligned}
(1 - \gamma) \nabla_{\pi_j} J^i(\pi) &= \mathbb{E} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{\tau} \left[r_0^i 1(j \in \{i\}) + \gamma r_1^i 1(j \in \text{Pa}^i(s_0^i)) \right. \right. \\
&\quad \left. \left. + \gamma^2 r_2^i 1(j \in \cup_{m \in \text{Pa}^i(s_1^i)} \text{Pa}^m(s_0^m)) + \gamma^3 Q^i(\mathbf{s}_3, \mathbf{a}_3) | \mathbf{s}_0, \mathbf{a}_0 \right] \right].
\end{aligned}$$

Also, notice that $\cup_{k \in \text{Pa}^i(s_1^i)} \text{Pa}^k(s_0^k)$ is the set of 2-hop parents of s_2^i on the realised trajectory τ .

In general, with trajectories of states $\tau = (\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2, \dots)$, by repeating the above steps, we can write the policy gradient as

$$(1 - \gamma) \nabla_{\pi_j} J^i(\pi) = \mathbb{E}_{\mathbf{s}_0 \sim \rho_{\pi}, \mathbf{a}_0 \sim \pi} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{\tau} \left[\sum_{t=0}^{\infty} \gamma^t r_t^i 1(j \in \text{Pa}^i(s_t^i, t, \tau)) | \mathbf{s}_0, \mathbf{a}_0 \right] \right],$$

where we define $\text{Pa}^i(s_t^i, k, \tau)$ the k -hop parents of s_t^i on the trajectory τ , that is

$$\begin{aligned}
\text{Pa}^i(s_t^i, k, \tau) &:= \text{Pa}^i(s_t^i, k-1, \tau) \cup \left(\bigcup_{m \in \text{Pa}^i(s_{t-k}^i, k-1, \tau)} \text{Pa}^m(s_{t-k}^m) \right) \\
&= \bigcup_{m \in \text{Pa}^i(s_{t-k}^i, k-1, \tau)} \text{Pa}^m(s_{t-k}^m),
\end{aligned} \tag{10}$$

where the last equality is because $i \in \text{Pa}^i(s_t^i), \forall s_t^i \in \mathcal{S}^i$. As the base case, we define the 0-hop parents of any state as $\text{Pa}^i(s_t^i, 0, \tau) = \{i\}$.

Finally, we can see that the sets $\{\text{Pa}^i(s_t^i, t, \tau)\}_t$ is a non-decreasing sequence, because (10) can also be written as

$$\text{Pa}^i(s_t^i, t, \tau) = \bigcup_{n \in \text{Pa}^i(s_{t-1}^i)} \text{Pa}^n(s_{t-1}^n, t-1, \tau), \tag{11}$$

and that $i \in \text{Pa}^i(s_{t-1}^i)$. As a result, if there exists a timestep $t' > 0$ where $j \in \text{Pa}^i(s_{t'}^i, t', \tau)$, then all subsequent timesteps $t'' \geq t'$, $j \in \text{Pa}^i(s_{t''}^i, t'', \tau)$. We can let t' be the smallest of such timesteps (in

which case $t' = T_{ji}(\tau, s_0^j)$, then

$$(1 - \gamma) \nabla_{\pi_j} J^i(\pi) = \mathbb{E}_{\rho_\pi, \pi} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_\tau \left[\sum_{k \geq t'} \gamma^k r^i(s_k^i, a_k^i) \mid \mathbf{s}_0, \mathbf{a}_0 \right] \right].$$

If there does not exist such a timestep, then we can simply let $\nabla_{\pi_j} J^i(\pi) = 0$ on this trajectory, which is equivalent to $t' = \infty$. $\square$

Proof of Lemma 4.6. For fixed agents i and j , and a joint policy π , define the dependency-truncated value functions as

$$Q_{\mathbf{P}, \mathcal{G}}^{i,j}(\mathbf{s}, \mathbf{a}) := \mathbb{E}_{\tau \sim \mathbf{P}, \pi} \left[\sum_{t=t'}^{\infty} \gamma^t r_t^i \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right] \quad (12)$$

$$= \mathbb{E}_{\tau \sim \mathbf{P}, \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t^i 1(j \in \text{Pa}^i(s_t^i, t, \tau)) \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right], \quad (13)$$

where we use $t' := T_{ji}(s_0^j, \tau)$ as a shorthand for the first timestep on trajectory τ at which there exists a directed path in graph $\mathcal{G}$ from the originating vertex (s_0, j) to $(s_{t'}, i)$; if no such time exists we take the Q value to be zero, and $\text{Pa}^i(s_t^i, k, \tau)$ denotes the set of k -hop parents in the path to (s_t, i) , i.e. $\text{Pa}^i(s_t^i, k, \tau) = \text{Pa}^i(s_t^i, k-1, \tau) \cup (\bigcup_{m \in \text{Pa}^i(s_t^i, k-1, \tau)} \text{Pa}^m(s_{t-k}^m))$ and $\text{Pa}^i(s_t^i, 0, \tau) = \{i\}$. Then,

$$\begin{aligned} & \nabla_{\pi_j} \mathcal{J}^i(\pi; \mathcal{G}) - \nabla_{\pi_j} \mathcal{J}^i(\pi; \mathcal{G}') \\ &= \mathbb{E}_{\rho_\pi, \pi} [\nabla \log \pi_j(a^j \mid s^j) (Q_{\mathbf{P}, \mathcal{G}}^{i,j}(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}, \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}))] \\ &= \mathbb{E}_{\rho_\pi, \pi} [\nabla \log \pi_j(a^j \mid s^j) (Q_{\mathbf{P}, \mathcal{G}}^{i,j}(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}', \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}) + Q_{\mathbf{P}', \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}, \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}))] \end{aligned}$$

Due to Proposition 4.1, we have $\nabla \log \pi_j(a^j | s^j) Q_{\mathbf{P}, \mathcal{G}}^{i,j}(\mathbf{s}, \mathbf{a}) = \nabla \log \pi_j(a^j | s^j) Q_{\mathbf{P}}^i(\mathbf{s}, \mathbf{a})$; where we denote $Q_{\mathbf{P}}$ the conventional Q value estimated on the transition kernel $\mathbf{P}$. As a result, using $\|\nabla \log \pi_j\| \leq B_j$ and a triangular inequality gives a standard bound

$$\begin{aligned} & \|\nabla_{\pi_j} \mathcal{J}(\pi; \mathcal{G}) - \nabla_{\pi_j} \mathcal{J}(\pi; \mathcal{G}')\| \\ & \leq B_j \left[\underbrace{\sup_{\mathbf{s}, \mathbf{a}} |Q_{\mathbf{P}}^i(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}'}^i(\mathbf{s}, \mathbf{a})|}_{T_1} + \underbrace{\sup_{\mathbf{s}, \mathbf{a}} |Q_{\mathbf{P}', \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}, \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a})|}_{T_2} \right] \end{aligned}$$

Bound on T_1 . The bound in T_1 is a standard result in model-based learning. In particular, from the simulation lemma (see e.g. Lemma 2.2 in Agarwal et al. (2019)), one obtains

$$\sup_{\mathbf{s}, \mathbf{a}} |Q_{\mathbf{P}}^i(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}'}^i(\mathbf{s}, \mathbf{a})| \leq \frac{\gamma \varepsilon R_{\max}}{(1 - \gamma)^2}.$$

Bound on T_2 . To bound T_2 , we observe that given a trajectory τ and a proper graph $\mathcal{G}'$, then the estimated graph-truncated values do not depend on the underlying kernel; the difference only presents in the expectation with the difference in probability of trajectories. We start from the recursive form of $Q_{\mathbf{P}, \mathcal{G}}^{i,j}$ in (13), as

$$\begin{aligned} Q_{\mathbf{P}, \mathcal{G}}^{i,j}(\mathbf{s}, \mathbf{a}) &= \mathbb{E}_{\tau \sim \mathbf{P}, \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t^i 1(j \in \text{Pa}^i(s_t^i, t, \tau)) \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right] \\ &= \mathbb{E}_{\tau \sim \mathbf{P}, \pi} \left[r_0^i 1(j \in \text{Pa}^i(s_0^i, 0, \tau)) + \sum_{t=1}^{\infty} \gamma^t r_t^i 1(j \in \text{Pa}^i(s_t^i, t, \tau)) \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right] \\ &= \mathbb{E}_{\tau \sim \mathbf{P}, \pi} \left[r_0^i 1(j \in \text{Pa}^i(s_0^i, 0, \tau)) + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r_t^i \left[1(j \in \text{Pa}^i(s_t^i, t-1, \tau)) \right. \right. \\ & \quad \left. \left. + 1 \left(j \in \bigcup_{k \in \text{Pa}^i(s_t^i, t-1, \tau)} \text{Pa}^k(s_0^k) \right) (1 - 1(j \in \text{Pa}^i(s_t^i, t-1, \tau))) \right] \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right] \\ &= \mathbb{E} \left[r_0^i 1(j \in \text{Pa}^i(s_0^i, 0, \tau)) + \gamma Q_{\mathbf{P}, \mathcal{G}}^{i,j}(\mathbf{s}_1, \mathbf{a}_1) + \gamma^{t'} r_{t'}^i \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right]. \end{aligned}$$

The last equality uses the definition of $Q^{i,j}$, and an observation that $1(j \in \cup_{k \in \text{Pa}^i(s_t^i, t-1, \tau)} \text{Pa}^i(s_0^k)) (1 - 1(j \in \text{Pa}^i(s_t^i, t-1, \tau))) = 1$ only if j is not in the $(t-1)$ -hop parents to s_t^i , but is in the t -hop parents, i.e. the definition of t' . Then,

$$\begin{aligned}
& Q_{\mathbf{P}, \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}) - Q_{\mathbf{P}', \mathcal{G}'}^{i,j}(\mathbf{s}, \mathbf{a}) \\
&= \mathbb{E}[r^i | \mathbf{s}, \mathbf{a}] 1(j \in \text{Pa}^i(s^i, 0)) + \gamma \sum_{\mathbf{s}'} \mathbf{P}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \sum_{\mathbf{a}'} \pi(\mathbf{a}' | \mathbf{s}') Q_{\mathbf{P}, \mathcal{G}'}^{i,j}(\mathbf{s}', \mathbf{a}') + \mathbb{E}_{\mathbf{P}, \pi}[\gamma^{t'} r_{t'} | \mathbf{s}, \mathbf{a}] \\
&\quad - \mathbb{E}[r^i | \mathbf{s}, \mathbf{a}] 1(j \in \text{Pa}^i(s^i, 0)) - \gamma \sum_{\mathbf{s}'} \mathbf{P}'(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \sum_{\mathbf{a}'} \pi(\mathbf{a}' | \mathbf{s}') Q_{\mathbf{P}', \mathcal{G}'}^{i,j}(\mathbf{s}', \mathbf{a}') - \mathbb{E}_{\mathbf{P}', \pi}[\gamma^{t'} r_{t'} | \mathbf{s}, \mathbf{a}] \\
&= \gamma \sum_{\mathbf{s}'} \left[\mathbf{P}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \sum_{\mathbf{a}'} \pi(\mathbf{a}' | \mathbf{s}') Q_{\mathbf{P}, \mathcal{G}'}^{i,j}(\mathbf{s}', \mathbf{a}') - \mathbf{P}'(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \sum_{\mathbf{a}'} \pi(\mathbf{a}' | \mathbf{s}') Q_{\mathbf{P}', \mathcal{G}'}^{i,j}(\mathbf{s}', \mathbf{a}') \right] \\
&\quad + \left[\mathbb{E}_{\mathbf{P}, \pi}[\gamma^{t'} r_{t'} | \mathbf{s}, \mathbf{a}] - \mathbb{E}_{\mathbf{P}', \pi}[\gamma^{t'} r_{t'} | \mathbf{s}, \mathbf{a}] \right].
\end{aligned}$$

From the definition of $Q_{\mathbf{P}, \mathcal{G}}^{i,j}$; it is easy to see that $0 \leq Q_{\mathbf{P}, \mathcal{G}}^{i,j} \leq \frac{R_{\max}}{1-\gamma}$, then the first term of the above equation can be bounded by

$$\gamma \|\mathbf{P}(\cdot | \mathbf{s}, \mathbf{a}) - \mathbf{P}'(\cdot | \mathbf{s}, \mathbf{a})\|_1 \frac{R_{\max}}{1-\gamma} \leq \frac{\gamma \epsilon R_{\max}}{1-\gamma}.$$

For the second term, we observe that the timestep t' of a given trajectory τ is the same regardless of whether the underlying transition probability is $\mathbf{P}$ or $\mathbf{P}'$, because we use the same graph $\mathcal{G}'$ and the paths on this graph are unchanged. As a result, we can think of having both $\mathbf{P}$ and $\mathbf{P}'$ to have the same reward function, but this reward function is non-Markovian on the original state space because rewards are only bestowed on the first timestep when agents i and j are connected. For that reason, we cannot directly apply the simulation lemma argument as in T_1 . Fortunately, this can easily be fixed by augmenting the state space so that the rewards are Markovian on this new state space. More specifically, for an agent j , we augment each state $\mathbf{s}$ with a vector of flags $\mathbf{f} \in \{0, 1\}^N$, indicating whether the agent j and other agents have already been connected. Let $\tilde{\mathbf{s}} = (\mathbf{s}, \mathbf{f})$. The augmented transition kernel can be defined as

$$\tilde{\mathbf{s}}_{t+1} = (\mathbf{s}_{t+1}, \mathbf{f}_{t+1}), \quad \mathbf{s}_{t+1} \sim \mathbf{P}(\cdot | \mathbf{s}_t, \mathbf{a}_t), \quad \mathbf{f}_{t+1}^i = 1 \left(i \in \bigcup_{k: \mathbf{f}_t^k = 1} \text{Pa}^k(s_t^k) \right), \quad \mathbf{f}_0^k = \begin{cases} 1 & k = j \\ 0 & k \neq j \end{cases}.$$

Under this augmented state space, the reward $r_{t'}$ is Markovian,

$$r_{t'}(\tilde{\mathbf{s}}_t, \mathbf{a}_t, \tilde{\mathbf{s}}_{t+1}) = r(\mathbf{s}_t, \mathbf{a}_t) 1(\mathbf{f}_t^i = 0 \wedge \mathbf{f}_{t+1}^i = 1),$$

Here, the indicator term denotes that the agent i is not connected to j in the current step but is in the next step. As a result, we can apply the same argument as in the bound T_1 by using the simulation lemma to conclude that

$$\sup_{\mathbf{s}, \mathbf{a}} \left| \mathbb{E}_{\mathbf{P}, \pi}[\gamma^{t'} r_{t'} | \mathbf{s}, \mathbf{a}] - \mathbb{E}_{\mathbf{P}', \pi}[\gamma^{t'} r_{t'} | \mathbf{s}, \mathbf{a}] \right| \leq \frac{\gamma \epsilon R_{\max}}{(1-\gamma)^2}.$$

Combining all the bounds, we get

$$\|\nabla_{\pi_j} \mathcal{J}(\pi; \mathcal{G}) - \nabla_{\pi_j} \mathcal{J}(\pi; \mathcal{G}')\| \leq \gamma \epsilon B_j R_{\max} \frac{3-\gamma}{(1-\gamma)^2} < \frac{3\gamma \epsilon B_j R_{\max}}{(1-\gamma)^2}.$$

□

Derivation of the inequality (7). Given the true graph $\mathcal{G}$ and an approximated graph $\mathcal{G}'$. The transition kernel $\mathbf{P}$ that is restricted to the graph structure of $\mathcal{G}'$ can be defined as the marginal distribution:

$$P_{\mathcal{G}'}^i(s^i | \mathbf{s}, \mathbf{a}) = P^i(s^{i'} | s^i, a^k : k \in \text{Pa}_{\mathcal{G}'}^i(s^i)) \quad \forall i \in [N].$$

Now we consider two possible scenarios with the mismatch between $\mathcal{G}$ and $\mathcal{G}'$:

1. $\mathcal{G}'$ contains an edge that does not exist in $\mathcal{G}$. In other word, $\exists s \in \mathcal{S}, i, j \in [N]$ such that $j \in \text{Pa}_{\mathcal{G}'}^i(s)$ and $j \notin \text{Pa}^i(s)$. In this case, since $\mathbf{P}_{\mathcal{G}}$ is the true transition kernel $\mathbf{P}$, conditioning $\mathbf{P}$ on additional substate-action pairs does not change the next states distributions. As a result, this has no effect on the distribution of $\mathbf{P}_{\mathcal{G}}$.
2. $\mathcal{G}$ contains an edge that does not exist in $\mathcal{G}'$. In this case, we are essentially predicting next states with missing information; $\mathbf{P}_{\mathcal{G}}$ is the average next states prediction marginalized on these missing edges.

As a result, we can safely ignore the former case and focus on the latter; we let for each state s^i , the set $\xi^i(s^i) = \text{Pa}^i(s^i) \setminus \text{Pa}_{\mathcal{G}'}^i(s^i)$. For simplicity, we assume that $|\xi^i(s^i)| = 1$; $\mathcal{G}$ differs from $\mathcal{G}'$ on only a single dependency agent at substate s^i . General cases can be easily expanded by a "bridging" argument; by gradually shrinking $\mathcal{G}$ to $\mathcal{G}'$ one edge at a time, which we will present later in the proof.

Let $m(s^i)$ be that missing agent, i.e. $\xi^i(s^i) = \{m(s^i)\}$. We then calculate the total variation of the difference between P^i and $P_{\mathcal{G}'}^i$ as

$$\begin{aligned}
& \mathbb{E}_{\mathbf{s}, \mathbf{a}} \|P^i(\cdot | \mathbf{s}, \mathbf{a}) - P_{\mathcal{G}'}^i(\cdot | \mathbf{s}, \mathbf{a})\|_1 \\
&= \mathbb{E}_{\mathbf{s}, \mathbf{a}} \left\| P^i \left(\cdot \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\} \cup \{(s^{m(s^i)}, a^{m(s^i)})\} \right) \right. \\
&\quad \left. - P^i \left(\cdot \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\} \right) \right\|_1 \\
&= \mathbb{E}_{s^i, a^i} \mathbb{E}_{(s, a) \sim i, -m(s^i)} \mathbb{E}_{s^{m(s^i)}, a^{m(s^i)}} \left\| P^i \left(\cdot \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\} \cup \{(s^{m(s^i)}, a^{m(s^i)})\} \right) \right. \\
&\quad \left. - P^i \left(\cdot \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\} \right) \right\|_1 \\
&\leq \sqrt{2} \mathbb{E}_{s^{m(s^i)}, a^{m(s^i)}} \left[D_{\text{KL}} \left[P^i \left(s^{i'} \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\} \cup \{(s^{m(s^i)}, a^{m(s^i)})\} \right) \right. \right. \\
&\quad \left. \left. \left\| P^i \left(s^{i'} \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s^i)\} \right) \right\|^{1/2} \right] \right] \\
&\leq \sqrt{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}} \left[I(S^{i'}; (S^{m(s^i)}, A^{m(s^i)}))^{1/2} \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s)\} \right] \\
&\leq \sqrt{2} \sum_j^N \mathbb{E}_{\mathbf{s}, \mathbf{a}} \left[I(S^{i'}; (S^j, A^j))^{1/2} \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}'}^i(s)\} \right],
\end{aligned}$$

where the expectations are over the joint state and action distributions (of an arbitrary policy). The first inequality is due to Pinsker's inequality, the second inequality is a combination of Jensen's inequality (with square root) and the mutual information relation to the KL divergence. The last inequality is due to the fact that $m(s^i) \in [N]$.

Now, for a general graph $\mathcal{G}'$, we construct a sequence of graphs $\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_N$, where any two consecutive graphs differ from each other at most one edge at each state s^i , i.e. $|\text{Pa}_{\mathcal{G}_n}^i \setminus \text{Pa}_{\mathcal{G}_{n+1}}^i(s^i)| \leq 1$. By triangular inequality,

$$\begin{aligned}
\mathbb{E} \|P^i - P_{\mathcal{G}'}^i\|_1 &\leq \mathbb{E} \|P^i - P_{\mathcal{G}_1}^i\|_1 + \dots + \mathbb{E} \|P_{\mathcal{G}_1}^i - P_{\mathcal{G}_N}^i\|_1 \\
&\leq \sqrt{2} \sum_n^N \mathbb{E} \left[\sum_j^N I(S^{i'}; (S^j, A^j))^{1/2} \mid \{(s^k, a^k); k \in \text{Pa}_{\mathcal{G}_n}^i(s)\} \right].
\end{aligned}$$

□

C.1 Reward Dependence Graph

In addition to state transitions, agents may also interact through their immediate rewards. To capture such interactions, a reward-dependence graph can be defined that specifies how agents' actions

influence one another’s instantaneous rewards. Formally, we allow the reward of agent i to depend on the state–action pairs of a subset of agents, denoted by $\text{Pa}_{\text{rw}}^i \subseteq [N]$. Similar to the state-dependence set of the transition probabilities, the reward function of agent i can be defined in the same way as

$$r^i(\{s_k, a_k \mid k \in \text{Pa}_{\text{rw}}^i(s_i)\}) \in [0, R_{\max}].$$

Also, in this section only, we will denote $\text{Pa}_{\text{st}}^i(\cdot)$ as the state dependence sets to distinguish it from the reward dependence sets explicitly.

Proposition C.2. *Fix a joint policy π . Let $i, j \in \mathcal{N}$. Let $\text{Pa}_{\text{rw}}^i(s_i)$ and $\text{Pa}_{\text{st}}^i(s_i)$ be the reward- and state-dependence agent set at state s_i of agent i , respectively. The policy gradient $\nabla_{\pi_j} \mathcal{J}^i(\pi)$ is given by*

$$\mathbb{E}_{\rho_{\pi}, \pi} \left[\nabla \log \pi_j(a_0^j | s_0^j) \mathbb{E}_{\tau} \left[\sum_{t=0}^{\infty} \gamma^t r_t^i 1(j \in \text{Pa}^i(s_t^i, t, \tau)) \mid \mathbf{s}_0, \mathbf{a}_0 \right] \right], \quad (14)$$

where $\text{Pa}^i(s_t^i, k, \tau)$ with $k > 0$ is the k -hop parents of s_t^i on the trajectory τ , defined as

$$\text{Pa}^i(s_t^i, k, \tau) = \bigcup_{m \in \text{Pa}^i(s_t^i, k-1, \tau)} \text{Pa}_{\text{st}}^m(s_{t-k}^m),$$

and we define the base-case of 0-hop parents as $\text{Pa}^i(s_t^i, 0, \tau) = \text{Pa}_{\text{rw}}^i(s_t^i)$. A path is defined according to the state dependence graph $\mathcal{G}$, similar to Proposition 4.1.

We note that the difference between this result and the result in 4.1 is the definition of k -hop parents. In particular, the 0-hop parents of an agent are initialized using the reward-dependence set, rather than the state-dependence graph alone. Proposition C.2 is strictly more general than proposition 4.1. Indeed, the 0-hop parent set defined in C is a special case of $\text{Pa}^i(s_t^i, 0, \tau)$ under the local reward settings $\text{Pa}_{\text{rw}}^i(s_t^i) = \{i\}$. The proof of Proposition C.2 follows identically to that of Proposition 4.1 and is therefore omitted. Intuitively, the reward dependence set captures the immediate effects and thus contributes directly to the 0-hop parents. However, longer-term influences arise through the state-dependence graph, as state transitions propagate the effects of agents’ states and actions into the future. Consequently, the parent sets are initialized by the reward-dependence sets and then expand recursively according to the state-dependence sets.

D Proof of Theorem 4.2

In this section, we let $\theta = (\theta_1, \theta_2, \dots, \theta_N)$ with each θ_i the parameters of agent i ’s policy. Additionally, we sometimes use π_j and π_{θ_j} interchangeably. We assume the following standard assumption on Lipschitzness and smoothness to hold for each agent.

Assumption D.1 (single-agent E-LS Yuan et al. (2022)). There exists constants $G, F > 0$ such that for every state $\mathbf{s}$ and every agent i ,

$$\mathbb{E}_{a^i \sim \pi_i} [\|\nabla_{\theta_i} \log \pi_i(a^i | s^i)\|^2] \leq G^2$$

$$\mathbb{E}_{a^i \sim \pi_i} [\|\nabla_{\theta_i}^2 \log \pi_i(a^i | s^i)\|] \leq F$$

Before presenting the proof, we want to note a critical detail that we glossed over in the main text. That is, at the timestep t , we do not have enough information to determine $T_{ji}(s_t^j)$. In simple terms, the meeting time (or equivalently the length of the path defined in C.1) depends on the sequence of future state trajectories that are yet to be observed at t . In formal terms, T_{ji} is not a stopping time. This will not be a problem if we sample the whole infinite trajectory $\tau_{0:\infty}$ and then consider the meeting time in hindsight. In a practical setting, we sample trajectories at a truncated H length. As a result, the meeting time that we consider in this setting should only consider paths that are determined before time H ; any interaction beyond the time horizon H will be dropped, that is, we consider such meeting times at H to be ∞ (or equivalent to saying they are unreachable), even though they can meet in a finite future. Formally, we define

$$T_{ji}^H(s_t^i, \mathbf{s}_{0:H}, \mathcal{G}) = \min(\{H \geq t' \geq t : j \in \text{Pa}^i(s_{t'}^i, t' - t, \mathbf{s}_{0:t'})\} \cup \{\infty\}). \quad (15)$$

Which is totally well-defined after observing the first H timesteps. Also, note that T_{ji}^H is an upper bound of T_{ji} , $T_{ji}^H(s_t^j, \tau) \geq T_{ji}(s_t^j, \tau), \forall t \leq H$. Then the m trajectories estimate of the dependent graph policy gradient can be defined based on this new meeting timestep,

$$\hat{\nabla}_{\theta_j^m} J^i(\theta, \mathcal{G}) = \frac{1}{m} \sum_{l=1}^m \sum_{t=0}^H \gamma^t \nabla_{\theta_j} \log \pi_j(a_{t,l}^j | s_{t,l}^j) \sum_{k=t+T_{ji}^H(s_{t,l}^j)}^H \gamma^k r_{k,l}^i.$$

Define $K_{ji}(n, \mathbf{s}_{0:n}) := \max\{k \leq n : j \in \text{Pa}^i(s_n^i, n-k, \mathbf{s}_{0:n})\} \leq n$ to be the last timestep at which the action has a consequence to timestep n , and $g_j^i(\tau)$ and $g_j^i(\tau, \mathcal{G})$ to be the single-trajectory estimate of traditional policy gradient and dependence graph policy gradient, respectively. For convenience, we also write $K_{ji}(n)$ when the (state) trajectory is obvious from the context. We then quantify the single-trajectory estimated gradient norms as

$$\begin{aligned} \mathbb{E}_{\tau} [\|g_j^i(\tau, \mathcal{G})\|^2] &= \mathbb{E} \left[\left\| \sum_{t=0}^{H-1} \nabla \log \pi_j(a_t^i | s_t^i) \sum_{k=t+T_{ji}^H}^{H-1} \gamma^k r_k^i \right\|^2 \right] \\ &= \mathbb{E} \left[\left\| \sum_{t=0}^{H-1} \gamma^t r_t^i \sum_{k=0}^{K_{ji}(t)} \nabla \log \pi_j(a_k^i | s_k^i) \right\|^2 \right] \\ &= \mathbb{E} \left[\left\| \sum_{t=0}^{H-1} \gamma^{t/2} r_t^i \gamma^{t/2} \sum_{k=0}^{K_{ji}(t)} \nabla \log \pi_j(a_k^i | s_k^i) \right\|^2 \right] \\ &\leq \mathbb{E} \left[\left(\sum_{k'=0}^{H-1} \gamma^{k'} (r_{k'}^i)^2 \right) \left(\sum_{t=0}^{H-1} \gamma^t \left\| \sum_{k=0}^{K_{ji}(t)} \nabla \log \pi_j(a_k^i | s_k^i) \right\|^2 \right) \right] \\ &\leq \frac{R_{\max}^2}{(1-\gamma)} \mathbb{E} \left[\left(\sum_{t=0}^{H-1} \gamma^t \left\| \sum_{k=0}^{K_{ji}(t)} \nabla \log \pi_j(a_k^i | s_k^i) \right\|^2 \right) \right] \\ &= \frac{R_{\max}^2}{(1-\gamma)} \sum_{t=0}^{H-1} \gamma^t \mathbb{E} \left[\sum_{k=0}^{K_{ji}(t)} \left\| \nabla \log \pi_j(a_k^i | s_k^i) \right\|^2 \right] \quad (\text{Lemma D.4}) \\ &\leq \frac{R_{\max}^2 G^2}{(1-\gamma)} \sum_{t=0}^{H-1} \gamma^t \mathbb{E}_{\tau} [K(t, \tau)] \end{aligned}$$

We can bound the final term as

$$\begin{aligned} \sum_{t=0}^{H-1} \gamma^t K_{ji}(t) &\leq \sum_{t=0}^{\infty} \gamma^t \sum_{k=0}^t 1(k \leq K_{ji}(t)) \\ &= \sum_{k=0}^{\infty} \sum_{t=k+T_{ji}(\tau, s_k^j)}^{\infty} \gamma^t \\ &= \sum_{k=0}^{\infty} \frac{\gamma^{k+T_{ji}(\tau, s_k^j)}}{1-\gamma} \end{aligned}$$

Note that we can now switch back to the meeting time in (1) when the time horizon $H = \infty$. Taking the expectation

$$\mathbb{E} \left[\sum_{t=0}^{H-1} \gamma^t K_{ji}(t) \right] \leq \frac{1}{1-\gamma} \sum_{k=0}^{\infty} \gamma^k \mathbb{E}_{\tau} [\gamma^{T_{ji}(\tau, s_k^j)}] \quad (16)$$

$$\leq \frac{\Gamma_{ji}}{(1-\gamma)^2} \quad (17)$$

where we define $\Gamma_j^i := \sup_{k, \pi} \mathbb{E}_\tau [\gamma^{T_{ji}(\tau, s_k^j)}] \leq 1$. Substitute everything back, and we get

$$\mathbb{E}_\tau [\|g_j^i(\tau, \mathcal{G})\|^2] \leq \frac{R_{\max}^2 G^2 \Gamma_j^i}{(1-\gamma)^3}$$

Finally, summing over all the cross gradient terms,

$$\begin{aligned} \mathbb{E}_\tau [\|g_j(\tau, \mathcal{G})\|^2] &= \mathbb{E}_\tau \left[\left\| \sum_i^N g_j^i(\tau, \mathcal{G}) \right\|^2 \right] \leq \sum_i^N \mathbb{E}_\tau [\|g_j^i(\tau, \mathcal{G})\|^2] \\ &\leq \frac{R_{\max}^2 G^2}{(1-\gamma)^3} \sum_i^N \Gamma_j^i. \end{aligned}$$

Then we have a bound on the stacked gradient estimate variance

$$\mathbb{E}_\tau [\|g(\tau, \mathcal{G})\|^2] := \mathbb{E}_\tau \left\| \begin{bmatrix} g_1(\tau, \mathcal{G}) \\ \vdots \\ g_N(\tau, \mathcal{G}) \end{bmatrix} \right\|^2 = \sum_{j=1}^N \mathbb{E}_\tau [\|g_j(\tau, \mathcal{G})\|^2] \leq \frac{R_{\max}^2 G^2}{(1-\gamma)^3} \sum_{j=1}^N \Gamma_j. \quad (18)$$

where we denote $\Gamma_j = \sum_i^N \Gamma_j^i$. Since $\hat{\nabla}_{\pi_j} J(\pi, \mathcal{G})$ and $\hat{\nabla}_{\pi_j} J(\pi)$ have the same expectation equals $\nabla_{\pi_j} J_H(\pi)$, the following results follow from Yuan et al. (2022), lemma 4.2, and (18),

$$\mathbb{E}_\tau [\|\hat{\nabla}_{\theta} J(\pi, \mathcal{G})\|^2] \leq \left(1 - \frac{1}{m}\right) \|\nabla_{\theta} J_H(\pi)\|^2 + \frac{G^2 R_{\max}^2}{m(1-\gamma)^3} \sum_j^N \Gamma_j \quad (19)$$

$$\mathbb{E}_\tau [\|\hat{\nabla}_{\theta} J(\pi)\|^2] \leq \left(1 - \frac{1}{m}\right) \|\nabla_{\theta} J_H(\pi)\|^2 + \frac{N^2 G^2 R_{\max}^2}{m(1-\gamma)^3} \quad (20)$$

where $\hat{\nabla}_{\pi_j} J(\pi, \mathcal{G}) = \frac{1}{m} \sum g_j(\tau, \mathcal{G})$ and $\hat{\nabla}_{\pi_j} J(\pi) = \frac{1}{m} \sum g_j(\tau)$, respectively. Then we have the following properties

$$\begin{aligned} \mathbb{E}_{\mathbf{a} \sim \pi} [\|\nabla_{\theta} \log \pi(\mathbf{a}|\mathbf{s})\|^2] &= \mathbb{E}_{\mathbf{a} \sim \pi} \left[\left\| \nabla_{\theta} \left(\sum_i^N \log \pi_i(a^i | s^i) \right) \right\|^2 \right] \\ &= \sum_i^N \mathbb{E}_{a^i \sim \pi_i} [\|\nabla_{\theta_i} \log \pi_i(a^i | s^i)\|^2] \leq N G^2 \end{aligned}$$

where the first equality is due to the independent structure of each agent's policy, and the second equality is due to the stacked structure of θ . Furthermore, since $\nabla_{\theta}^2 \log \pi(\mathbf{a}|\mathbf{s})$ is a block diagonal matrix, its norm equals its largest diagonal block spectral norm,

$$\begin{aligned} \mathbb{E}_{\mathbf{a} \sim \pi} [\|\nabla_{\theta}^2 \log \pi(\mathbf{a}|\mathbf{s})\|] &= \mathbb{E}_{\mathbf{a} \sim \pi} \left[\max_i \|\nabla_{\theta_i}^2 \log \pi_i(a^i | s^i)\| \right] \\ &\leq \sum_i^N \mathbb{E}_{a^i \sim \pi_i} [\|\nabla_{\theta_i}^2 \log \pi_i(a^i | s^i)\|] \leq N F \end{aligned}$$

This suggests that single-agent E-LS (D.1) implies joint E-LS. However, we can additionally utilise the dependence graph structure of the Networked MDP to obtain a tighter bound on the smoothness coefficient of the objective $J(\theta)$.

Lemma D.2. Under assumption D.1, $J(\theta)$ is L -smooth, namely $\|\nabla_{\theta}^2 J(\theta)\| \leq L$, with

$$L_i = \frac{R_{\max} \max_{j \leq N} \sum_i^N \Gamma_j^i}{(1-\gamma)^2} (G^2 + F).$$

Proof. Denote $g_{ji}(\tau, \mathcal{G}) = \sum_{t=0}^{\infty} \gamma^t r_t^i \left(\sum_{k=0}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right)$, $\psi_j(\tau) = \sum_{t=0}^{\infty} \nabla_{\theta_j} \log \pi_{\theta_j}(a_t^j | s_t^j)$ and $h_{ji}(\tau, \mathcal{G}) = \sum_{t=0}^{\infty} \gamma^t r_t^i \left(\sum_{k=0}^{K_{ji}(t)} \nabla_{\theta_j}^2 \log \pi_{\theta_j}(a_k^j | s_k^j) \right)$, then

$$\begin{aligned} \nabla_{\theta}^2 J^i(\theta) &= \nabla_{\theta} \mathbb{E}_{\tau} \left(\begin{bmatrix} g_{1i}(\tau, \mathcal{G}) \\ \vdots \\ g_{Ni}(\tau, \mathcal{G}) \end{bmatrix} \right) \\ &= \underbrace{\mathbb{E}_{\tau} \left[\begin{bmatrix} \psi_1(\tau) \\ \vdots \\ \psi_N(\tau) \end{bmatrix} \begin{bmatrix} g_{1i}(\tau, \mathcal{G}) \\ \vdots \\ g_{Ni}(\tau, \mathcal{G}) \end{bmatrix}^{\top} \right]}_{A^i} + \underbrace{\mathbb{E}_{\tau} \begin{bmatrix} h_{1i}(\tau, \mathcal{G}) & & \\ & \ddots & \\ & & h_{Ni}(\tau, \mathcal{G}) \end{bmatrix}}_{B^i} \end{aligned}$$

Then the Hessian of the total reward has the form

$$\nabla_{\theta}^2 J(\theta) = \sum_i A^i + \sum_i B^i$$

We can bound $\|\sum_i B^i\|$ by

$$\begin{aligned} \left\| \sum_i B^i \right\| &= \left\| \begin{bmatrix} \sum_i \mathbb{E}_{\tau} h_{1i}(\tau, \mathcal{G}) & & \\ & \ddots & \\ & & \sum_i \mathbb{E}_{\tau} h_{Ni}(\tau, \mathcal{G}) \end{bmatrix} \right\| \\ &= \max_{j \leq N} \left\| \sum_i^N \mathbb{E}_{\tau} h_{ji}(\tau, \mathcal{G}) \right\| \\ &\leq \max_{j \leq N} \sum_i^N \mathbb{E}_{\tau} [\|h_{ji}(\tau, \mathcal{G})\|] \\ &= \max_{j \leq N} \sum_i^N \mathbb{E}_{\tau} \left[\left\| \sum_{t=0}^{\infty} \gamma^t r_t^i \sum_{k=0}^{K_{ji}(t)} \nabla_{\theta_j}^2 \log \pi_{\theta_j}(a_k^j | s_k^j) \right\| \right] \\ &\leq \max_{j \leq N} \sum_i^N \mathbb{E}_{\tau} \left[\sum_{t=0}^{\infty} \gamma^t r_t^i \sum_{k=0}^{K_{ji}(t)} \left\| \nabla_{\theta_j}^2 \log \pi_{\theta_j}(a_k^j | s_k^j) \right\| \right] \\ &\leq R_{\max} \max_{j \leq N} \sum_i^N \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau} \left[\sum_{k=0}^{K_{ji}(t)} \left\| \nabla_{\theta_j}^2 \log \pi_{\theta_j}(a_k^j | s_k^j) \right\| \right] \\ &\leq R_{\max} F \max_{j \leq N} \sum_i^N \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau} [K_{ji}(t)] \\ &\leq \frac{R_{\max} F \max_{j \leq N} \sum_i^N \Gamma_j^i}{(1 - \gamma)^2} \end{aligned}$$

where the final inequality follows similarly to (17), the second inequality is due to the triangular inequality, and the penultimate inequality is due to the assumption D.1 that holds for each state and by lemma D.4. Now we need to bound A norm, first observe that A is a block diagonal matrix due to lemma D.5,

$$A = \mathbb{E}_{\tau} \begin{bmatrix} \sum_i^N g_{1i}(\tau) \psi_1(\tau)^{\top} & & \\ & \ddots & \\ & & \sum_i^N g_{Ni}(\tau) \psi_N(\tau)^{\top} \end{bmatrix}$$

As a result, we can bound the norm of A by

$$\begin{aligned}
\|A\| &= \left\| \begin{bmatrix} \mathbb{E}_\tau \sum_i^N g_{1i}(\tau) \psi_1(\tau)^\top & & \\ & \ddots & \\ & & \mathbb{E}_\tau \sum_i^N g_{Ni}(\tau) \psi_N(\tau)^\top \end{bmatrix} \right\| \\
&= \max_{j \leq N} \left\| \mathbb{E}_\tau \sum_i^N g_{ji}(\tau) \psi_j(\tau)^\top \right\| \\
&\leq \max_{j \leq N} \mathbb{E}_\tau \sum_i^N \sum_t^\infty \gamma^t r_t^i \left\| \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \sum_{t'}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j)^\top \right\| \\
&\leq R_{\max} \max_{j \leq N} \sum_i^N \sum_t^\infty \gamma^t \mathbb{E} \left\| \sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right\|^2 \\
&\leq R_{\max} G^2 \max_{j \leq N} \sum_i^N \sum_t^\infty \gamma^t \mathbb{E}[K_{ji}(t)] \\
&\leq \frac{R_{\max} G^2 \max_{j \leq N} \sum_i^N \Gamma_j^i}{(1-\gamma)^2}.
\end{aligned}$$

□

Without the dependence graph, then L can scale with N due to the scale of the joint E-LS. Then by Corollary 4.7 in Yuan et al. (2022) applied for the joint parameter vector θ , by choosing stepsize $\eta = \frac{\epsilon^2 m}{2L\nu}$, minibatch size m between 1 and $\frac{2\nu}{\epsilon^2}$, horizon H to $O(\log(N/\epsilon)/\log(1/\gamma))$, and number of iterations $T = \frac{8\delta_0 L\nu}{m\epsilon^4}$ where $\delta_0 = J^* - J(\theta_0) \leq \frac{NR_{\max}}{1-\gamma}$, we have the sample complexity result of

$$\begin{aligned}
Tm \times H &= O\left(\frac{\delta_0 L\nu \log(N/\epsilon)}{\log(1/\gamma)\epsilon^4}\right) \\
&= \begin{cases} O\left(\frac{NR_{\max}^4 \Gamma \max_j \Gamma_j \log(N/\epsilon)}{(1-\gamma)^6 \epsilon^4 \log(1/\gamma)}\right) & \text{using dependence graph PG} \\ O\left(\frac{N^3 R_{\max}^4 \max_j \Gamma_j \log(N/\epsilon)}{(1-\gamma)^6 \epsilon^4 \log(1/\gamma)}\right) & \text{using vanilla PG} \end{cases} \quad (21)
\end{aligned}$$

to guarantee a convergence to stationary points $\mathbb{E}[\|\nabla_{\theta^U} J(\pi_{\theta^U})\|^2] \leq O(\epsilon^2)$, where $\nu = \frac{G^2 R_{\max}^2 \sum_j^N \Gamma_j}{(1-\gamma)^3}$ as in (19) when using dependence graph policy gradient, and $\nu = \frac{G^2 R_{\max}^2 N^2}{(1-\gamma)^3}$ as in (20) when using traditional policy gradient.

We consider a simpler version of the gradient domination condition in the literature (Agarwal et al., 2019; Yuan et al., 2022) for illustrative purposes.

Assumption D.3 (Gradient domination). There exists a constant $\mu > 0$ such that, for all θ ,

$$\|\nabla_{\theta_j} J(\theta)\| \geq \mu \left(\sup_{\pi_j} J(\pi_{\theta^U}^{-j}, \pi^j) - J(\theta) \right) \quad \forall j \in [N].$$

Then it is easy to see that $O(\epsilon^2) \geq \mathbb{E}[\|\nabla_{\theta^U} J(\pi_{\theta^U})\|^2] = \mathbb{E}\left[\sum_j^N \|\nabla_{\theta_j^U} J(\pi_{\theta^U})\|^2\right] \geq \sum_j^N \left(\mathbb{E}\left[\|\nabla_{\theta_j^U} J(\pi_{\theta^U})\|\right]\right)^2$, so each $\mathbb{E}\|\nabla_{\theta_j^U} J(\pi_{\theta^U})\| \leq O(\epsilon)$. Combine this with assumption D.3, we have

$$\mathbb{E}[J(\theta^U)] \geq \sup_{\pi_j} \mathbb{E}[J(\pi_{\theta^U}^{-j}, \pi^j)] - O(\epsilon), \quad \forall j \in [N].$$

Lemma D.4. For all non-negative integers $t \geq 0$,

$$\mathbb{E}_\tau \left[\left\| \sum_{t=0}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_t^j | s_t^j) \right\|^2 \right] = \mathbb{E}_\tau \left[\sum_{t=0}^{K_{ji}(t)} \left\| \nabla_{\theta_j} \log \pi_{\theta_j}(a_t^j | s_t^j) \right\|^2 \right]$$

Proof. The difficulty of this lemma is the dependence of $K_{ji}(t, \tau)$ to the trajectory τ . More specifically,

$$\begin{aligned} \mathbb{E}_\tau \left[\left\| \sum_{k=0}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right\|^2 \right] &= \mathbb{E}_\tau \left[\sum_{k=0}^{K_{ji}(t)} \left\| \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right\|^2 \right] \\ &+ \mathbb{E}_\tau \left[\sum_{k=0}^{K_{ji}(t)} \sum_{k' \neq k}^{K_{ji}(t)} \left\langle \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j), \nabla_{\theta_j} \log \pi_{\theta_j}(a_{k'}^j | s_{k'}^j) \right\rangle \right]. \end{aligned}$$

Here $K_{ji}(t)$ is measurable with respect to $\mathcal{F}_t$, while the gradients $\nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j)$ are measurable with respect to $\mathcal{F}_k \subset \mathcal{F}_t$. In other word, $K_{ji}(t)$ is not a stopping time w.r.t. $\mathcal{F}_k$. To circumvent this problem, let $S_{0:t} = \sigma(\mathbf{s}_0, \mathbf{s}_1, \dots, \mathbf{s}_t)$ denote the σ -algebra generated by the state trajectory up to time t . By the tower property of conditional expectation:

$$\begin{aligned} \mathbb{E}_\tau \left[\sum_{k=0}^{K_{ji}(t)} \sum_{k' \neq k}^{K_{ji}(t)} \left\langle \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j), \nabla_{\theta_j} \log \pi_{\theta_j}(a_{k'}^j | s_{k'}^j) \right\rangle \right] \\ = 2\mathbb{E}_{\mathbf{s}_{0:t}} \mathbb{E}_{\mathbf{a}_{0:t}} \left[\sum_{k=0}^{K_{ji}(t)} \sum_{k' > k}^{K_{ji}(t)} \left\langle \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j), \nabla_{\theta_j} \log \pi_{\theta_j}(a_{k'}^j | s_{k'}^j) \right\rangle \middle| S_{0:t} \right] \quad (22) \end{aligned}$$

Recall that $K_{ji}(t, \mathbf{s}_{0:t}) = \max\{k \leq t : j \in \text{Pa}^i(s_t^i, t - k, \mathbf{s}_{0:t})\}$; the meeting time determined entirely by the state trajectory upto timestep t , hence $K_{ji}(t)$ is $S_{0:t}$ -measurable; Conditioned on $S_{0:t}$, $K_{ji}(t)$ is a deterministic constant. Then

$$\begin{aligned} (22) &= 2\mathbb{E}_{\mathbf{s}_{0:t}} \sum_{k=0}^{K_{ji}(t)} \sum_{k' > k}^{K_{ji}(t)} \mathbb{E}_{\mathbf{a}_{0:k'-1}} \left[\left\langle \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j), \underbrace{\mathbb{E}_{\mathbf{a}_{k'}^j} [\nabla_{\theta_j} \log \pi_{\theta_j}(a_{k'}^j | s_{k'}^j) | s_{k'}^j]}_{=0} \right\rangle \middle| S_{0:t} \right] \\ &= 0. \end{aligned}$$

□

Lemma D.5. For any agents i, j, j' . Let

$$(*) := \mathbb{E}_\tau \left[\sum_t^\infty \sum_{t'}^\infty \gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \nabla_{\theta_{j'}} \log \pi_{\theta_{j'}}(a_{t'}^{j'} | s_{t'}^{j'})^\top \right],$$

then $(*) = 0$ if $j \neq j'$. And when $j = j'$, then

$$(*) = \mathbb{E}_\tau \left[\sum_t^\infty \gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \left(\sum_{t'}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j) \right)^\top \right].$$

Proof. We first consider the case when $j \neq j'$ and $j' \neq i$. For any timestep t and t' , similar to the proof of Lemma D.4, we condition on the state trajectory up to a timestep $\max(t, t')$. Define

$$\begin{aligned} (**) &:= \mathbb{E} \left[\gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \nabla_{\theta_{j'}} \log \pi_{\theta_{j'}}(a_{t'}^{j'} | s_{t'}^{j'})^\top \middle| \mathbf{s}_{0:\max(t, t')} \right] \\ &= \mathbb{E} \left[\gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \underbrace{\mathbb{E}_{\mathbf{a}_{t'}^{j'}} [\nabla_{\theta_{j'}} \log \pi_{\theta_{j'}}(a_{t'}^{j'} | s_{t'}^{j'}) | s_{t'}^{j'}]}_{=0}^\top \middle| \mathbf{s}_{0:\max(t, t')} \right] \\ &= 0 \end{aligned}$$

where we can integrate over the gradient of agent j' because j' and j are different agents, so their gradients are independent, and the rewards r^i only depend on agent $i \neq j'$.

Next, if $j \neq j'$ and $j' = i$, then

$$\begin{aligned}
(**) &= \mathbb{E} \left[\gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \nabla_{\theta_i} \log \pi_{\theta_i}(a_{t'}^i | s_{t'}^i)^\top \middle| \mathbf{s}_{0:\max(t,t')} \right] \\
&= \mathbb{E} \left[\gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \underbrace{\mathbb{E}_{a_k^j} [\nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) | s_k^j]}_{=0} \right) \nabla_{\theta_i} \log \pi_{\theta_i}(a_{t'}^i | s_{t'}^i)^\top \middle| \mathbf{s}_{0:\max(t,t')} \right] \\
&= 0,
\end{aligned}$$

where we can integrate over all a_k^j because given $s_t^i, r_t^i(s_t^i, a_t^i)$ is independent from all a_k^j with $j \neq i$. Combining the two above results, and summing over all timesteps t and t' , then we have when $j \neq j'$,

$$(*) = \sum_t \sum_{t'}^{(\infty)} (**) = 0$$

Finally, we consider the diagonal case when $j = j'$, then

$$\begin{aligned}
(*) &= \mathbb{E} \left[\sum_t \gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \sum_{t'} \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j)^\top \right] \\
&= \mathbb{E} \left[\sum_t \gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \sum_{t'}^t \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j)^\top \right] \\
&= \mathbb{E} \left[\sum_t \gamma^t r_t^i \left(\sum_k^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_k^j | s_k^j) \right) \sum_{t'}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j)^\top \right]
\end{aligned}$$

where the last equality is because from proposition 4.1,

$$\begin{aligned}
0 &= \nabla_{\theta_j} J^i(\boldsymbol{\theta}) - \nabla_{\theta_j} J^i(\boldsymbol{\theta}, \mathcal{G}) \\
&= \mathbb{E}_\tau \left[\sum_t \gamma^t r_t^i \sum_{t'}^t \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j) \right] - \mathbb{E}_\tau \left[\sum_t \gamma^t r_t^i \sum_{t'}^{K_{ji}(t)} \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j) \right] \\
&= \mathbb{E}_\tau \left[\sum_t \gamma^t r_t^i \sum_{t'=T_{ji}(t)+1}^t \nabla_{\theta_j} \log \pi_{\theta_j}(a_{t'}^j | s_{t'}^j) \right],
\end{aligned}$$

□

E Environment and reward specifications

Star-Spread MPE is an environment that is inspired by the spread MPE benchmark (Lowe et al., 2017). There are N agents and N landmarks. Agent 0 is the *hub*; agents $1, \dots, N-1$ are *leaves*. Landmarks are placed on a circle of radius 0.7 at evenly spaced angles. Each agent starts at a uniformly random position in $[-1, 1]^2$ and takes one of five discrete actions (no-op, up, down, left, right) per step. The episode horizon is $T_{\text{ep}} = 50$.

Denote by $p_i \in \mathbb{R}^2$ the position of agent i and by ℓ_j the j -th landmark position. Let $d_{ij} := \|p_i - \ell_j\|_2$ be the agent-to-landmark distance. The per-agent reward at each step is

$$r_0^{\text{hub}} = - \sum_{j=1}^N \min_i d_{ij} + \xi, \quad r_i^{\text{leaf}} = - \min_j d_{ij} + \xi \quad \text{for } i = 1, \dots, N-1, \quad (23)$$

where $\xi \sim \mathcal{N}(0, 1)$ are sampled independently from standard Gaussian noise to increase the difficulty of the credit assignment problem. The hub is charged the team coverage loss (the sum of each landmark’s distance to its nearest agent), while each leaf is only charged its own distance to the closest landmark. In the local-reward setting, the per-agent rewards in Eq. (23) are delivered individually; in the global-reward setting, the scalarized team reward $R_t = \sum_i r_t^i$ is delivered to every agent.

The hub observes $[p_0, \text{all } p_i, \text{all } \ell_j] \in \mathbb{R}^{4N+2}$ (centralized observation), whereas each leaf observes $[p_i, \text{all } \ell_j] \in \mathbb{R}^{2N+2}$ (its own position and the landmark map). The state-dependence graph used as the *oracle* graph is

$$G = \max(I_N, \mathbf{1}e_0^\top), \quad \text{equivalently, } G[i, j] = \mathbf{1}\{i = j\} \vee \mathbf{1}\{j = 0\}, \quad (24)$$

i.e. column 0 and the diagonal are all ones. Semantically, the hub’s next state depends on all other agents’ states and actions, and every agent’s next state depends on its own current state. No leaf–leaf edges are present.

Level-Based Foraging Christianos et al. (2020) is a suite of several gridworld-based environments, where the agents navigate to collect random spawned food across the map. Both agents and food have a level each, and the food is only collected if the sum of the levels of agents collecting it is larger than or equal to the level of the food. Rewards are assigned to agents upon collection based on their level of contribution to the collecting process.

Additionally, we consider a variation of the LBF benchmark. In this modified version, we introduce a more challenging reward structure that makes it harder for cooperative behavior among agents to emerge. Specifically, while the original LBF environment rewards each agent individually based on their level when collecting food, we adopt a winner-takes-all reward scheme: only the agent with the highest level receives a reward. In cases where multiple agents share the highest level, we break ties in a *deterministic* manner. Furthermore, the food levels are set so that at least two agents must cooperate to collect them (–coop setup). This setup penalizes myopic agents, while having no impact on approaches that rely on a global reward signal. We run our experiments on 3 scenarios from vanilla LBF and 3 scenarios with winner-take-all reward setups.

SMAClite Michalski et al. (2023) is a fully open-source implementation of the popular SMAC benchmark Samvelyan et al. (2019). SMAClite allows us to modify the environment to extract the agent-specific reward, which SMAC does not support. More specifically, the damage rewards are now attributed directly to the source agents that cause the damage. Other reward signals that cannot be attributed to a single source, for example, winning rewards and enemies’ self-damage, are distributed evenly to all agents. Upon killing an enemy, the agents are rewarded with a large reward. As a result, while each agent is rewarded for the damage inflicted on the enemies, the local reward scheme can encourage the agents to "steal" kills from others. We run our experiments on 6 selected scenarios, ranging from large team coordination, such as 25m_vs_30m, to heterogeneous, role-specific scenarios like 3s5z_vs_3s6z.

E.1 Experimental Settings and Network Architecture

We adopt our backbone algorithms on both MAPPO Yu et al. (2022) and IPPO De Witt et al. (2020) with Dependence Graph (DG). We compare our approach against the same algorithms under two other reward settings, namely global and local rewards. Furthermore, we also compare our method to IQL (local reward), QMIX (global reward) Rashid et al. (2020), and QPLEX (global reward) Wang et al. (2020) as other strong value-based baselines from both global and local reward settings. For a fair comparison, all methods utilize parameter sharing between agents.

All the algorithms are based on the PyMARL framework Papoudakis et al. (2020). As a result, all the hyperparameters used in this paper closely follow those reported in the framework. For a fair comparison, all PPO-based algorithms with different reward setups are GAE-enabled, with the GAE lambda λ set to 0.95 as recommended in Schulman et al. (2015). Since SMAClite is not tested in the PyMARL, we follow suit SMAClite original paper Michalski et al. (2023) to use the same hyperparameters of PyMARL for SMAC. For QMIX and QPLEX, we adopt the finetuned hyperparameters for SMAC, as reported in Hu et al. (2021), and used it in SMAClite. The full set of hyperparameters used in PPO-based methods is provided as follows.

All neural networks used in our experiments for the baseline methods, including the policy and value networks, follow the original PyMARL implementations without modification. In our method, the

	SMAClite	LBF	MPE
hidden dimension	64	128	64
learning rate	0.0005	0.0005	0.0005
reward standardisation	False	False	True
network type	GRU	FC	FC
entropy coefficient	0.001	0.001	0.01
clip eps	0.2	0.2	0.2

Table 1: All PPO-based methods, including our MAPPO-DG and IPPO-DG, use the same hyperparameters for each benchmark.

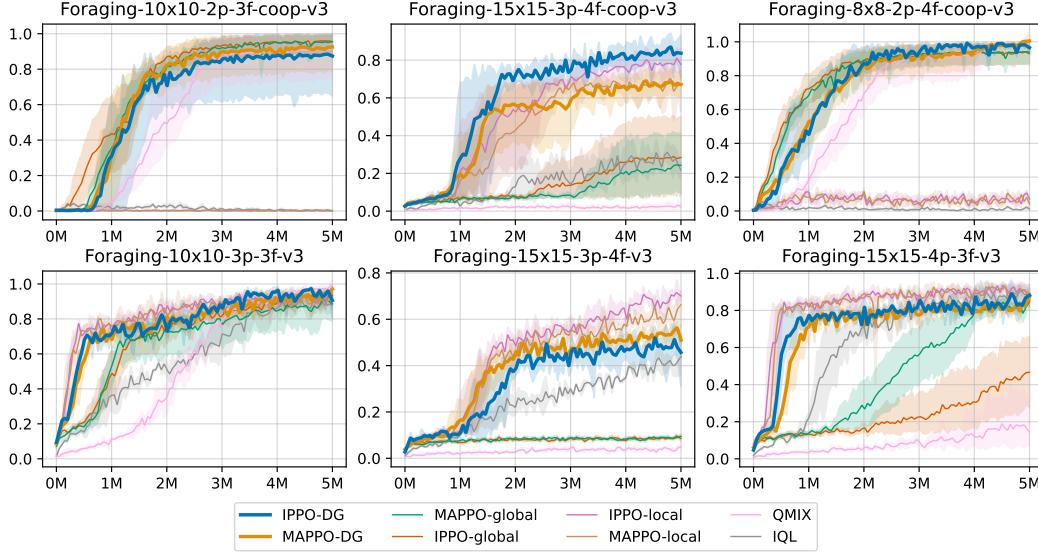


Figure 8: Full results on the LBF benchmark.

state encoder E is implemented as a three-layer neural network with hidden dimensions of 64 and an output dimension of 64. The single-agent reverse world model q_φ , the action predictor model q_ϕ , and the multi-agent reverse world model q_ψ share the same architecture: a four-layer neural network with 256 neurons in the first hidden layer and 128 neurons in each of the remaining hidden layers. All networks use ReLU activations and Layer Normalization Ba et al. (2016).

E.2 Detailed Experiment Results

Figure 8 presents the complete results corresponding to Figure 5 in the main paper. We can see that in the vanilla LBF scenarios (bottom row), agents trained with local reward signals consistently outperform those using global rewards. Especially in the 15x15-3p-4f environment, where all global methods, including MAPPO-global, QMIX, and IPPO-global struggle significantly due to the sparsity of the reward signals. In the 15x15-4p-3f setting, the addition of extra agents alleviates the sparsity issue to some extent, possibly due to the parameter sharing mechanism. MAPPO-global is able to converge to an optimal policy, albeit at a slower rate and with higher variance compared to the local reward setup. QMIX, on the other hand, fails to learn effectively even in this slightly less sparse scenario. IQL using local reward in this setting outperforms QMIX with global rewards on all vanilla LBF setups except for in Foraging-10x10-3p-3f, where they both converge to optimal policies, but IQL learns a bit slower. In contrast, the winner-take-all with the cooperative setup (top row) poses challenges to the local reward approach. In this setup, due to the competitive nature of the reward function, the local reward approach struggles; it fails to learn in 2 out of 3 scenarios. Interestingly, some of the completely different branches of algorithms behave quite similarly under the same reward setting. In particular, in 10x10-2p-3f-coop and 8x8-2p-4f-coop, all local reward algorithms fail, including on-policy methods with both MAPPO and IPPO, value-based method IQL, and value decomposition QMIX. In 15x15-3p-4f-coop, we observe a similar trend as in the winner-take-all

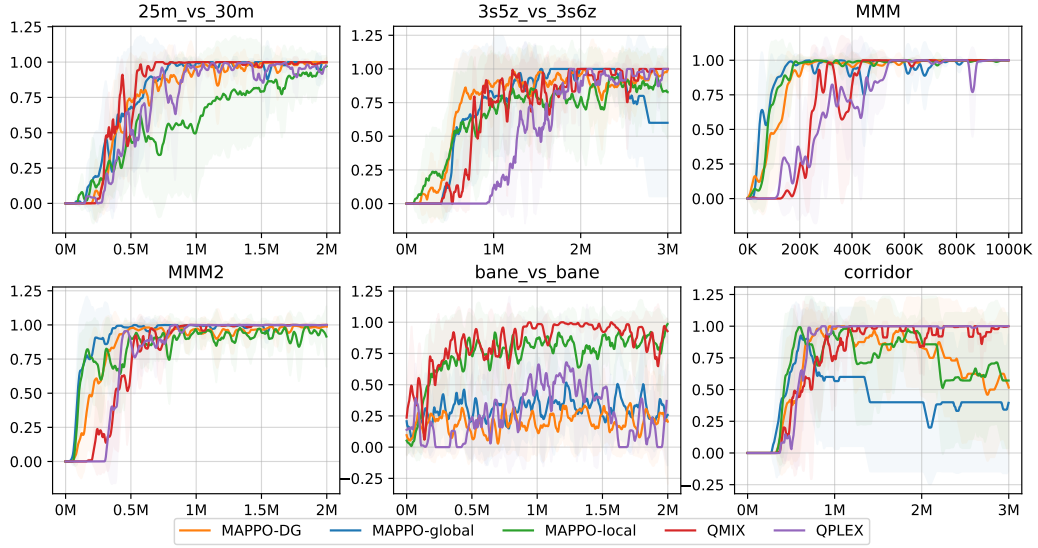


Figure 9: Full results on the SMAClite benchmark.

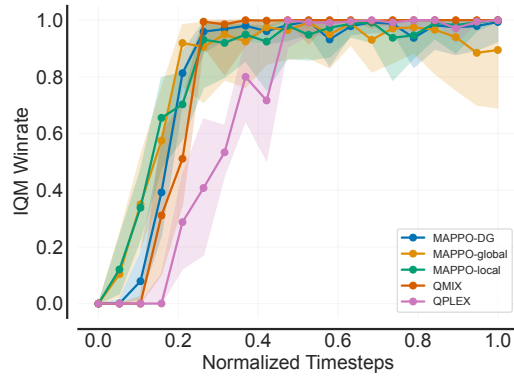


Figure 10: Result on the SMAClite benchmark on 6 scenarios.

setup, where an additional agent improves the performance of local reward with both value-based and on-policy methods, except for QMIX. On the other hand, our method remains competitive across all the scenarios, demonstrating its robustness across different setups.

Figures (9) and (10) present the full results on the SMAClite benchmark. QMIX achieves consistently strong performance across all scenarios. QPLEX performs quite similarly to QMIX, but slower in 3s5z_vs_3s6z and does not learn in bane_vs_bane. In most scenarios, our method and global MAPPO achieve comparable performance; for example, in 25m_vs_30m, both outperform local MAPPO. On corridor, however, our method and local MAPPO outperform global MAPPO. We attribute the unstable learning behavior observed in all MAPPO-based methods to suboptimal hyperparameters adopted from PyMARL.

E.3 Algorithm

Algorithm 2 Multi-Agent PPO (MAPPO and IPPO) with Dependence Graph.

```

1: Initialize shared actor parameters  $\theta$ , centralized critic parameters  $\vartheta$ , encoder  $E$ , single-agent
   reverse world model  $q_\varphi$ , action prediction model  $q_\phi$ , multi-agent reverse world model  $q_\psi$ 
2: for iteration = 1 to  $N_{\text{iter}}$  do
3:   Initialize empty buffer  $\mathcal{D}$ 
4:   for episode = 1 to  $N_{\text{episodes}}$  do
5:     for  $t = 1$  to  $T$  do
6:       for each agent  $i = 1, \dots, N$  do
7:         Sample action  $a_t^i \sim \pi_\theta(\cdot \mid o_t^i)$ 
8:       end for
9:       Execute joint action  $\mathbf{a}_t = (a_t^1, \dots, a_t^N)$ 
10:      Observe rewards  $\{r_t^i\}$ , next state  $s_{t+1}$ , next obs  $\{o_{t+1}^i\}$ , done flag  $d_t$ 
11:      Store  $(s_t, o_t^i, a_t^i, r_t^i, \log p_t^i, d_t, s_{t+1})$  in  $\mathcal{D}$  for all  $i$ 
12:    end for
13:  end for
14:  for each timestep  $t$  in  $\mathcal{D}$  do
15:    Initialize Adjacency matrix  $\mathbf{A}_t = \text{diag}(1)$ 
16:    for each agent  $i$  in  $\mathcal{N}$  do
17:      Compute  $V_t^i = V_\vartheta^i(s_t)$  if MAPPO and  $V_\vartheta^i(o_t^i)$  if IPPO
18:      Compute latent state  $z_t^i = E(o_t^i)$ 
19:      Compute action distribution entropy  $H_1 = H(q_\phi(z_t^i))$ 
20:      for each agent  $j$  in  $\mathcal{N}$  do
21:        Compute latent state  $z_t^j = E(o_t^j)$ 
22:        Compute latent next state  $z_{t+1}^j = E(o_{t+1}^j)$ 
23:        Compute multi-agent reverse model distribution entropy  $H_2 = H(q_\psi(z_t^i, z_t^j, z_{t+1}^j))$ 
24:        Compute edge  $\mathbf{A}_{ij} = 1(H_2/H_1 < c)$ 
25:      end for
26:    end for
27:  end for
28:  Compute dependence truncated Advantage  $\{A_t^i\}_{t=0}^{T-1}$  in Algorithm 1 using  $\{V_t^i\}_{t=0}^{T-1}$  and
    $\{\mathbf{A}_t^i\}_{t=0}^{T-1}$ 
29:  for epoch = 1 to  $K$  do
30:    for each minibatch  $\mathcal{B}$  do
31:      for each sample  $(o_t^i, a_t^i, \log p_{\text{old}}^i, A_t^i)$  in  $\mathcal{B}$  do
32:         $\log p^i = \log \pi_\theta(a_t^i \mid o_t^i)$ 
33:         $r^i = \exp(\log p^i - \log p_{\text{old}}^i)$ 
34:         $L_{\text{clip}}^i = \min(r^i A_t^i, \text{clip}(r^i, 1 - \epsilon, 1 + \epsilon) A_t^i)$ 
35:      end for
36:       $L_{\text{actor}} = -\frac{1}{|\mathcal{B}|} \sum_i L_{\text{clip}}^i$ 
37:       $L_{\text{critic}} = \frac{1}{|\mathcal{B}|} \sum (V_\vartheta^i(s) - V_{\text{target}}^i)^2$  if MAPPO and  $\frac{1}{|\mathcal{B}|} \sum (V_\vartheta^i(o^i) - V_{\text{target}}^i)^2$  if IPPO
38:       $L_{\text{entropy}} = -\beta \mathbb{E}[\mathcal{H}(\pi_\theta(\cdot \mid o^i))]$ 
39:       $L = L_{\text{actor}} + c_v L_{\text{critic}} + L_{\text{entropy}}$ 
40:      Update  $\theta, \vartheta$  using gradients of  $L$ 
41:    end for
42:  end for
43:  Compute latent states  $\{z_t^i\}_{t,i} = \{E(o_t^i)\}$ 
44:  Compute Encoder loss  $L_E = \frac{1}{|\mathcal{D}||\mathcal{N}|} \sum_{i,t} \text{CE}(q_\varphi(z_t^i, z_{t+1}^i), a_t^i)$ 
45:  Update Encoder  $E, \varphi$  using gradient of  $L_E$ 
46:  Detach gradient in  $\{z_t^i\}_{t,i}$ 
47:  Compute reverse model loss  $L_{\text{rv}} = \frac{1}{|\mathcal{D}||\mathcal{N}|^2} \sum_{i,j,t} [\text{CE}(q_\phi(z_t^i), a_t^i) + \text{CE}(q_\psi(z_t^i, z_t^j, z_{t+1}^j), a_t^i)]$ 
48:  Update  $\phi, \psi$  using gradient of  $L_{\text{rv}}$ 
49: end for

```
